Thundercomm
AIOT ENABLER



2023-01-31 9:16:44 AM CST
Steven_cam.moorvision.com

Thundercomm TurboX C410/C610 Open Kit Linux Embedded

Software User Manual

Rev. V3.2.1
July 14, 2023

DN: tc-gle-23321

Revision History

Revision	Date	Author	Description
1.0	Aug 15, 2020	Xiaojun Wang	Initial release.
2.0	Apr 01, 2021	Xiaojun Wang	• Update Display port. • Add Digital zoom. • Add Lens distortion correction. • Add Video. • Add Face detection. • Add Sleep mode.
3.0	Dec 10, 2021	Xiaojun Wang	• Update Figure 1-1. • Add Table 1-1. • Update the following sections: ▫ 2.2. SD card ▫ 2.4. Display Port ▫ 2.6. USB 3.1 Type-A ▫ 2.9. Wi-Fi/BT ▫ 2.12.2. Push video stream to PC via adb tool ▫ 2.12.4. Live snapshot ▫ 2.12.7. Lens distortion correction ▫ 2.13.3. Multiplex streaming ▫ 2.16. SNPE ▫ 2.18. Sleep mode • Add the following new sections: ▫ 2.10. Gyro and accelerator sensor ▫ 2.14.3. Capture with Gstreamer ▫ 2.14.4. Capture with parecord ▫ 2.19. OTA upgrade
3.1	Apr 01, 2022	Xiaojun Wang	• Update Figure 1-1 and Figure 1-2. • Add Table 1-1. • Update the following sections: ▫ 2.3. DSI LCD panel ▫ 2.11.5. Dual camera concurrency (1080P recording) ▫ 2.12.1. Video recording • Remove *Section 2.5 Touch panel*. • Add a note in 2.16. Face detection. • Fix format problems.

Thundercomm Confidential

Copyright© 2023 Thundercomm Technology Co., Ltd. All rights reserved.

Revision	Date	Author	Description
3.2	Apr 03, 2023	Xiaojun Wang	Update this document to make it compatible with LE2.0: • Update OS support info. in Chapter 1. Product Overview and Table 1-2. • Update the following sections: ▫ 2.3. DSI LCD panel ▫ 2.4. DisplayPort ▫ 2.6. LED control ▫ 2.7. Buttons ▫ 2.8. Wi-Fi/BT ▫ 2.10.1. WAN Mode ▫ 2.11.1. Environment configuration ▫ 2.11.5. Dual camera concurrency (1080P recording) ▫ 2.11.6. Digital zoom ▫ 2.12.4. Video file playback on LCD ▫ 2.12.5. HDMI IN
3.2.1	July 14, 2023	Ranran Cong	Update the following sections: • 2.8. Wi-Fi/BT • 2.11.4. Live snapshot • 2.11.6. Digital zoom • 2.12.5. HDMI IN

Thundercomm Confidential
2025-01-31 9:16:44 AM CST
Stefan_ucam.moorvision.com

Copyright© 2023 Thundercomm Technology Co., Ltd. All rights reserved.

Table List

Table 1-1. Board layout

Table 1-2. Features and specifications

Table 2-1. Button description

Thundercomm Confidential
2025-01-31 9:16:44 AM CST
Stefan_ucam.moorvision.com

Copyright© 2023 Thundercomm Technology Co., Ltd. All rights reserved.

About This Document

- Illustrations in this documentation might look different from your product.
- Depending on the model, some optional accessories, features, and software programs might not be available on your device.
- Depending on the version of operating systems and programs, some user interface instructions might not be applicable to your device.
- Documentation content is subject to change without notice. Thundercomm makes constant improvements on the documentation of the products, including this guidebook.
- Function declarations, function names, type declarations, attributes, and code samples appear in a different format, for example, `cp armcc armcpp`.
- Code variables appear in angle brackets, for example, `<number>`.
- Button, tool, and key names appear in bold font, for example, click **Save** or press **Enter**.
- Commands to be entered appear in a different font; on the host computer use $ as shell prompt, while on the target device use # as shell prompt, for example,

```
$ adb devices
# logcat
```

- Part of the code that does not contain instructions appear in a different format, for example,

```
SUBSYSTEM=="usb", ATTR{idVendor}=="18d1", MODE="0777", GROUP="adm"
```

- Folders and files are formatted in italic, for example, *turbox_flash_flat.sh*.

Thundercomm Confidential
2025-01-31 9:16:44 AM CST
Stefan_ucam.moorvision.com

Copyright© 2023 Thundercomm Technology Co., Ltd. All rights reserved.

Table of Contents

Chapter 1. Product Overview ..... - 2 -

1.1. Features and specifications ..... - 3 -

1.2. Product diagram ..... - 3 -

Chapter 2. Function Test ..... - 4 -

2.1. Debug port ..... - 4 -

2.2. SD card ..... - 5 -

2.3. DSI LCD panel ..... - 5 -

2.4. DisplayPort ..... - 8 -

2.5. USB 3.1 Type-A ..... - 9 -

2.6. LED control ..... - 10 -

2.7. Buttons ..... - 11 -

2.8. Wi-Fi/BT ..... - 12 -

2.9. Sensor ..... - 17 -

2.10. Ethernet ..... - 19 -

2.11. Camera ..... - 21 -

2.12. Video ..... - 27 -

2.13. Audio ..... - 31 -

2.14. TensorFlow lite ..... - 36 -

2.15. SNPE ..... - 36 -

2.16. Face detection (for LE1.0 only) ..... - 38 -

2.17. Sleep mode ..... - 39 -

2.18. OTA upgrade ..... - 40 -

Appendix 1. Notices ..... - 43 -

Appendix 2. Trademarks ..... - 45 -

Thundercomm Confidential
2025-01-31 9:16:44 AM CST
Stefan_ucam.modnvision.com

Chapter 1. Product Overview

The TurboX C410/C610 Open Kit is a cost-effective solution to evaluate the performance and build the prototype of the smart cameras and intelligent IoT devices quickly. The C410/C610 Open Kit features rich interfaces, including USB Type A, HDMI in, Ethernet, and camera mezzanine-board expansions enabling the Kit "open" for building camera-based devices, including panorama, smart camera, and video conference camera. The C410/C610 Open Kit supports Linux LE1.0 and LE2.0, LA1.0 (Android 10) and LA2.0 (Android 12) operating systems, combining powerful artificial intelligence and machine learning to address increasing demands for smart cities, commercial and enterprise, homes, as well as vehicle applications.

➲ **NOTE:** "TurboX" referred to herein is the English text of our registered trademark TURBOX.

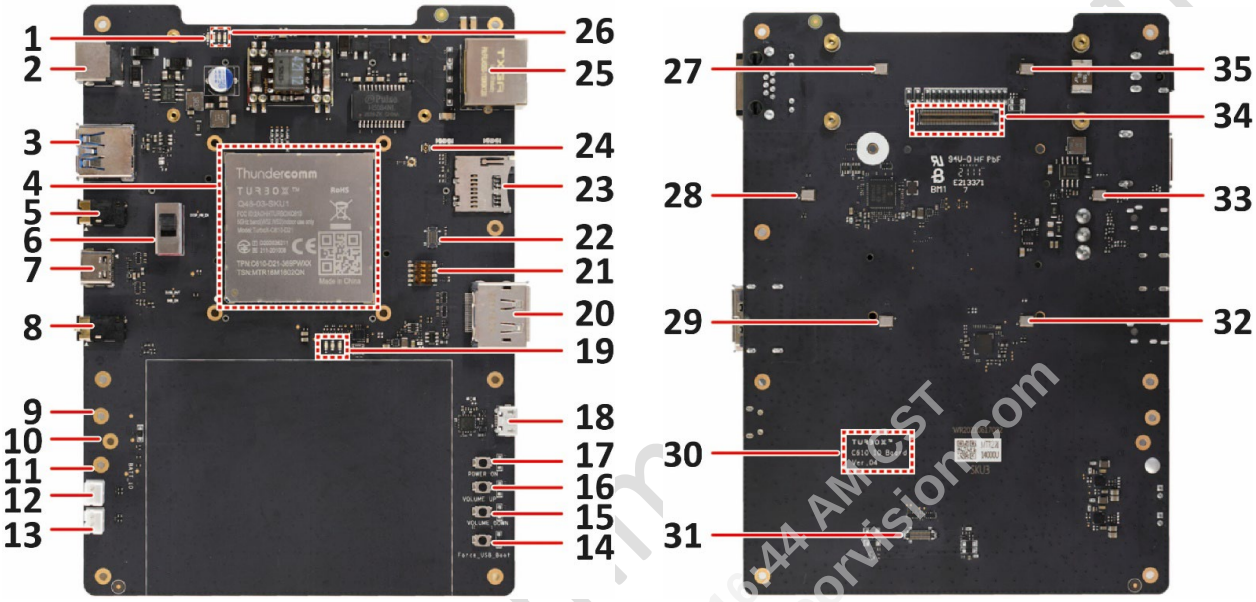


Figure 1-1. Board Layout (Hardware Version 04)

Table 1-1. Board layout

1. Power LED	13. Speaker connector 2	25. Ethernet connector
2. 12V DC in jack	14. Force USB Boot button	26. PMIC RGB LED
3. USB 3.1 Type-A connector (USB 2.0 protocol only)	15. Volume down button	27. Digital microphone 1
4. Heat sink fixing hole	16. Volume up button	28. Digital microphone 2
5. Line in connector	17. Power on button	29. Digital microphone 3
6. Power source selective switch	18. UART debug connector	30. Hardware version
7. USB 3.1 Gen1 Type-C connector	19. GPIO LED	31. LCD connector
8. Headset connector	20. DisplayPort	32. Digital microphone 4
9. VBAT + pad	21. Boot config switch	33. Digital microphone 5
10. BAT thermal pad	22. JTAG debug connector	34. Camera mezzanine board connector
11. VBAT - pad	23. Micro SD card slot	35. Digital microphone 6
12. Speaker connector 1	24. Wi-Fi/BT antenna connector	-

1.1. Features and specifications

Table 1-2. Features and specifications

Category	Description
System	TurboX C410/C610 SOM
Vertical Mezzanine Board (Camera board)	• Vertical A: Panorama - 2 x IMX577 • Vertical B: Conference: - 1 x IMX415 + 1 x HDMI IN
Display Interfaces	1 x DP; 1 x 5.0 TFT-LCD
Audio Interfaces	• 1 x 3.5mm headset • 6 x digital microphone • 2 x speaker out
General interfaces on mother board	• 1 x USB 3.1 Gen1 Type-C • 1 x USB 2.0 Type A • 1 x Micro USB (for debug) • 1 x SD card • 1 x GbE Ethernet • 1 x ACCELEROMETER+GYRO Sensor • 1 x Wi-Fi/BT antenna connector • 6 x LED • 4 x button (Volume up/down, Force USB Boot, Power on)
Power Supply	DC IN 12V, POE 15W, Battery Connector
Operating Environment	• Operation temperature: -25℃ ~ 75℃ • Operation humidity: 5%~95%, non-condensing
Dimension	120 x 250 mm
OS Support	• Linux Embedded 1.0 • Linux Embedded 2.0

1.2. Product diagram

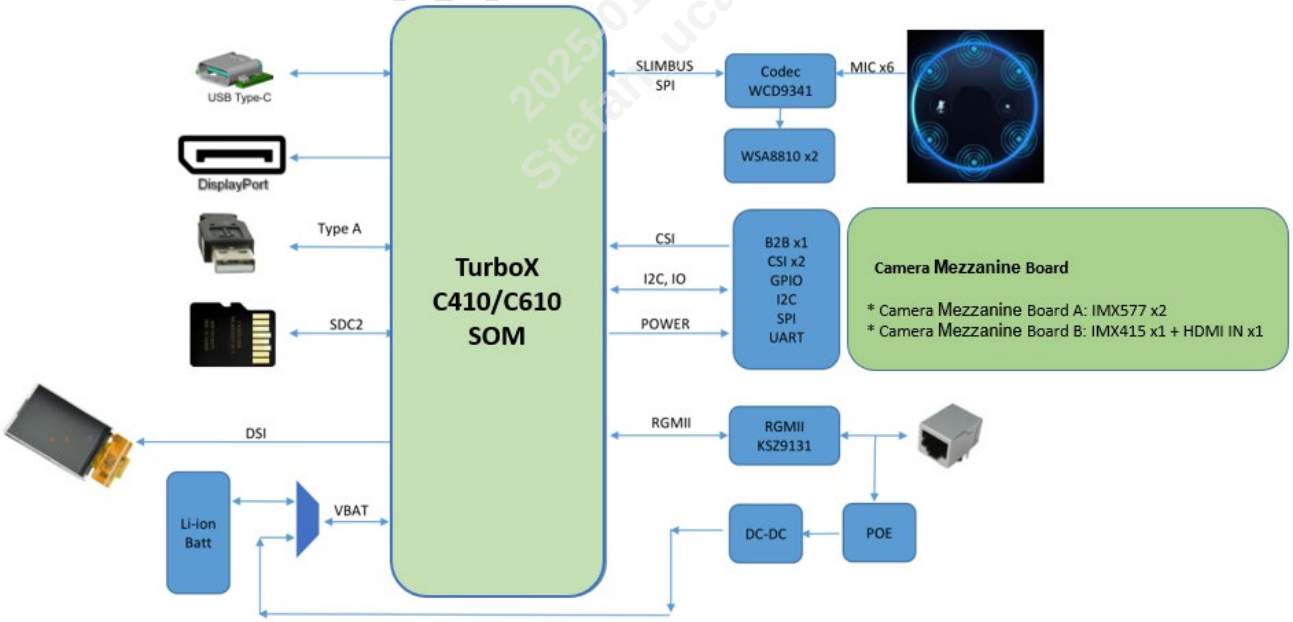


Figure 1-2. Block Diagram

Thundercomm Confidential
2025-01-31 9:16:44 AM CST
Stefan.ucam.moorvision.com

Chapter 2. Function Test

This chapter introduces detailed testing steps for various board functions.

2.1. Debug port

Step 1. Connect the board to PC via the **UART debug connector** (No. 18 in Figure 1-1).

Step 2. Use a UART tool you prefer, for example, **Minicom**.

A new serial port will be added to your computer after the device is connected. Select the new serial port, and configure it to 115200 8N1. Refer to Figure 2-1 for the configuration of **Minicom 2.7**.



Figure 2-1. Minicom 2.7 Configuration

Step 3. Log in to the board system with the following account credentials.

```
… …
[  OK  ] Started start dsp variants.
[  OK  ] Reached target Multi-User System.
         Starting Update UTMP about System Runlevel Changes...
[  OK  ] Started Update UTMP about System Runlevel Changes.

qsap 202001010057 qcs610-odk ttyMSM0
qcs610-odk login: root
Password: oelinux123
```

➲ **NOTE:** UART debug cannot be used in perf/user version.

Thundercomm Confidential 2025-01-31 9:16:14 AM CST Stefan_c.cam.mobivision.com

2.2. SD card

Step 1. SD card is hot-swappable. When inserted to the board, SD card shall automatically mount to /mnt/*sdcard/* to access contents.

```
$ adb root && adb shell
# df -h
Filesystem               Size      Used Available Use% Mounted on
/dev/mmcblk1p1         119.3G      5.6G    113.7G   5% /mnt/sdcard
```

Step 2. Run the following commands to test read and write speed of the SD card:

```
$ adb root && adb shell
# cd /mnt/sdcard
# dd if=/dev/urandom of=test.txt bs=30M count=2 conv=fsync
dd if=/dev/urandom of=test.txt bs=30M count=2 conv=fsync
2+0 records in
2+0 records out
62914560 bytes (60.0MB) copied, 6.366916 seconds, 9.4MB/s
# echo 3 >/proc/sys/vm/drop_caches
# dd if=/mnt/sdcard/test.txt of=/dev/null
dd if=/sdcard/test.txt of=/dev/null
122880+0 records in
122880+0 records out
62914560 bytes (60.0MB) copied, 0.092965 seconds, 645.4MB/s
```

2.3. DSI LCD panel

The board supports two LCD panel versions: ili9881c and ili9881p.



Figure 2-2. ili9881c



Figure 2-3. ili9881p

Thundercomm Confidential
2025-01-31 9:16:44 AM CST
Stefan_ucam.moorvision.com

➲ **NOTES:**

- Software versions below LE1.0.Post-CS1.r002008 of LE1.0 baseline can only support ili9881c.
- LE1.0.Post-CS1.r002008 and later versions of LE1.0 baseline as well as all software versions of LE2.0 can support both ili9881c and ili9881p.
- To use ili9881p, it is required to connect FPC. Refer to the pin sequence marked on the FPC to ensure proper connection.

Step 1. Connect the LCD panel to the board via the **LCD connector** (No. 31 in Figure 1-1). Refer to Figure 2-2 and Figure 2-3.

Step 2. Download the corresponding CDT file of the desired LCD panel.

For detailed procedure, refer to *TurboX C410 C610 SOM_Flash CDT Image with TurboX Easy Flash User Guide*.

- **CDT file of ili9881c**: turbox-c610-cdt-opk_LCD_ili9881c
- **CDT file of ili9881p:** turbox-c610-cdt-opk_LCD_ili9881p

Step 3. Connect debug UART and press the **Power on button** (No. 17 in Figure 1-1) to boot up the system normally.

Refer to the corresponding log output.

```
Format: Log Type - Time(microsec) - Message - Optional Info
Log Type: B - Since Boot(Power On Reset),  D - Delta,  S - Statistic
S - QC_IMAGE_VERSION_STRING=BOOT.XF.3.1-00569-SM6150LZB-1
S - IMAGE_VARIANT_STRING=SDM6150LA
S - OEM_IMAGE_VERSION_STRING=8b8891cc45fb
…
S -    123623 - PBL, End
B -    143136 - SBL1, Start
…
UEFI Start     [ 1024] SEC
ASLR        : ON
DEP         : ON (RTB)
…
Loader Build Info: Oct 22 2020 19:44:02
VB: RWDeviceState: Succeed using devinfo!
Total DDR Size: 0x000000007ED00000
KeyPress:0, BootReason:0
Fastboot=0, Recovery:0
GetVmData: No Vm data present! Status = (0x3)
VM Hyp calls not present
Booting from slot (_a)
Booting Into Mission Mode
....
Update Device Tree total time: 77 ms
Start EBS        [13803]
BDS: LogFs sync skipped, Unsupported
App Log Flush : 6 ms
Exit EBS        [13823] UEFI End
[    0.000000] Booting Linux on physical CPU 0x0
[    0.000000] Linux version 4.14.117+ (daniel@TSBJ-FA-PC-03418) (clang version
8.0.12 for Android NDK) #17 SMP PREEMPT Wed Oct 21 16:03:34 CST 2020
[    0.000000] Boot CPU: AArch64 Processor [51df805e]
[    0.000000] Machine: IOT
[    0.000000] earlycon: msm_geni_serial0 at MMIO 0x0000000000880000 (options '')
```

Thundercomm Confidential
2025-01-21 9:16:44 AM CST
Stefan ucam.moonvision.com

Step 4.Test the LCD panel with **Weston**. The LCD panel is successfully connected with flowers appearing on the screen.

1) Disable */dev/dm-0 write-protected*.

```
$ adb root
$ adb disable-verity
$ adb reboot
$ adb root && adb remount && adb shell mount -o remount,rw /
```

2) Input the following commands to output flowers covering the LCD screen.

- LE1.0:

```
$ adb shell
# export XDG_RUNTIME_DIR=/dev/socket/weston
# weston --tty=1 --idle-time=123456 &
```

- LE2.0:

```
$ adb shell
$ vi /etc/xdg/weston/weston.ini
[output]
name=Virtual-1
mode=off
[output]
name=DSI-1
mode=on
[output]
name=DP-1
mode=off

# export WESTON_DISABLE_ATOMIC=1
# export XDG_RUNTIME_DIR=/run/user/root
# weston --tty=1 --idle-time=123456 &
```

3) Execute the following commands and the camera stream preview will be displayed on the LCD.

- **LE1.0:**

```
$ adb shell
# export XDG_RUNTIME_DIR=/dev/socket/weston
# gst-launch-1.0 -e qtiqmmfsrc name=qmmf camera=0 ! "video/x-
raw(memory:GBM),format=NV12,width=1920,height=1080,framerate=30/1" !
waylandsink sync=false fullscreen=true
```

- **LE2.0:**

```
$ adb shell
# export XDG_RUNTIME_DIR=/run/user/root
# gst-launch-1.0 qtiqmmfsrc name=camsrc ! video/x-raw\(memory:GBM\),
format=NV12, width=1920,height=1080, framerate=30/1 ! waylandsink
fullscreen=true async=true sync=false
```

4) Exit **Weston**.

```
# killall weston
```

2.4. DisplayPort

➲ **NOTE:** It is recommended to use DP-DP cable; DP-HDMI cable may cause incompatibility problems.

2.4.1. DisplayPort test on LE1.0

If there is no code compilation environment, refer to 2.12.2. Multiplex decoding (for LE1.0 only) for more information on DisplayPort settings.

Alternatively, follow the steps below to compile the **modetest** tool and test the DisplayPort.

Step 1.Set up the **modetest** on the Yocto project.

1) Export machine distro.

```
$ cd <project_root>/apps_proc/
$ MACHINE=qcs610-odk-64 DISTRO=qti-distro-fullstack-virtualization-debug
VARIANT=debug
$ source poky/qti-conf/set_bb_env.sh
```

2) Modify `local.conf`.

```
$ vim conf/local.conf
  - INHERIT += "rm_work"
  + #INHERIT += "rm_work"
```

3) Build `libdrm.`

```
$ bitbake -fc cleanall libdrm ; bitbake libdrm
```

4) Set up the **modetest** on the device

```
$ cd  tmp-glibc/work/aarch64-oe-linux/libdrm/2.4.83-r0/package
$ adb push usr/bin/modetest /usr/bin
$ adb shell
# chmod 777 /usr/bin/modetest
```

Step 2.Test external display monitor with **modetest**.

- Obtain the panel information.

```
# modetest -M msm_drm | more > log.txt
# vi log.txt
Encoders:
id      crtc    type    possible crtcs  possible clones
27      0       DSI     0x00000007      0x00000000
43      0       Virtual 0x00000007      0x00000000
52      0       TMDS    0x00000007      0x00000000
57      0       DPMST   0x00000007      0x00000000
58      0       DPMST   0x00000007      0x00000000
....

530    connected DP-1           510x290        12   52
  modes:
  name refresh (Hz) hdisp hss hse htot vdisp vss vse vtot)
  1920x1080 60 1920 2008 2052 2200 1080 1084 1089 1125 flags: phsync, pvsync;
type: preferred, driver
  1600x900 60 1600 1624 1704 1800 900 901 904 1000 flags: phsync, pvsync; type:
driver
  1280x1024 75 1280 1296 1440 1688 1024 1025 1028 1066 flags: phsync, pvsync;
type: driver
  1280x1024 60 1280 1328 1440 1688 1024 1025 1028 1066 flags: phsync, pvsync;
type: driver
  1152x864 75 1152 1216 1344 1600 864 865 868 900 flags: phsync, pvsync; type:
driver
  1024x768 75 1024 1040 1136 1312 768 769 772 800 flags: phsync, pvsync; type:
driver
  1024x768 60 1024 1048 1184 1344 768 771 777 806 flags: nhsync, nvsync; type:
....
```

- Test display function with **modetest**, the monitor will show the color bar.

```
# modetest -M msm_drm -s 53:1920x1080
setting mode 1920x1080-60Hz@XR24 on connectors 53, crtc 107
```

2.4.2. DisplayPort test on LE2.0

Enter the following commands to output flowers covering the DP monitor.

```
$ adb shell
$ vi /etc/xdg/weston/weston.ini
[output]
name=Virtual-1
mode=off
[output]
name=DSI-1
mode=off
[output]
name=DP-1
mode=on

# export WESTON_DISABLE_ATOMIC=1
# export XDG_RUNTIME_DIR=/run/user/root
# weston --tty=1 --idle-time=123456 &
```

2.5. USB 3.1 Type-A

Step 1. Connect a mouse to the board via the **USB 3.1 Type-A connector** (No.3 in Figure 1-1) and check the information of the input device.

```
$ adb shell
# dmesg -c
[  469.069224] msm-dwc3 a800000.hsusb: DWC3 exited from low power mode
[  469.330682] usb 1-1: new low-speed USB device number 2 using xhci-hcd
[  469.497240] usb 1-1: New USB device found, idVendor=04ca, idProduct=0061
[  469.504821] usb 1-1: New USB device strings: Mfr=1, Product=2, SerialNumber=0
[  469.522175] usb 1-1: Product: USB Optical Mouse
[  469.527070] usb 1-1: Manufacturer: PixArt
[  469.579764] input: PixArt USB Optical Mouse as
/devices/platform/soc/a800000.hsusb/a800000.dwc3/xhci-hcd.0.auto/usb1/1-1/1-
1:1.0/0003:04CA:0061.0001/input/input2
[  469.650707] hid-generic 0003:04CA:0061.0001: input: USB HID v1.11 Mouse [PixArt
USB Optical Mouse] on usb-xhci-hcd.0.auto-1/input0

/*According to the log ,it's  create a input device at '/dev/input/event2'*/
# cat /dev/input/event2 | hexdump
0000000 8b0c 5f1f 4a16 000c 0002 0000 ffff ffff
0000010 8b0c 5f1f 4a16 000c 0000 0000 0000 0000
0000020 8b0c 5f1f 68fc 000c 0002 0000 fff9 ffff
0000030 8b0c 5f1f 68fc 000c 0000 0000 0000 0000
0000040 8b0c 5f1f 883d 000c 0002 0000 fff5 ffff
0000050 8b0c 5f1f 883d 000c 0002 0001 0002 0000
0000060 8b0c 5f1f 883d 000c 0000 0000 0000 0000
0000070 8b0c 5f1f a781 000c 0002 0000 fffd ffff
f1f a781 000c 0002 0001 0001 0000
```

Step 2.Connect a USB storage device and check the device information.

```
# dmesg | grep usb
[  597.589443] msm-dwc3 a800000.hsusb: DWC3 exited from low power mode
[  598.920681] usb 1-1: new high-speed USB device number 3 using xhci-hcd
[  599.078146] usb 1-1: New USB device found, idVendor=0781, idProduct=5588
[  599.086489] usb 1-1: New USB device strings: Mfr=2, Product=3, SerialNumber=1
[  599.094176] usb 1-1: Product: USB Extreme Pro
[  599.098700] usb 1-1: Manufacturer: SanDisk
[  599.103199] usb 1-1: SerialNumber: 12345778C5FD
[  599.129923] usb-storage 1-1:1.0: USB Mass Storage device detected
[  599.148015] scsi host0: usb-storage 1-1:1.0
[  600.168565] scsi 0:0:0:0: Direct-Access     SanDisk  Extreme Pro      0    PQ: 0
ANSI: 6
[  600.202613] sd 0:0:0:0: [sda] Write Protect is off
[  600.207624] sd 0:0:0:0: [sda] Mode Sense: 43 00 00 00
[  600.276102]  sda: sda1
```

2.6. LED control

You can control the LED status with the following commands.

```
$ adb shell
# echo 0 > /sys/devices/platform/soc/soc:leds-gpios/leds/led_bt/brightness
//LE1.0/LE2.0 Turn off
# echo 1 > /sys/devices/platform/soc/soc:leds-gpios/leds/led_bt/brightness
//LE1.0/LE2.0 Turn on
# echo 0 > /sys/devices/platform/soc/soc:leds-gpios/leds/led_wifi/brightness
//LE1.0/LE2.0 Turn off
# echo 1 > /sys/devices/platform/soc/soc:leds-gpios/leds/led_wifi/brightness
//LE1.0/LE2.0 Turn on
# echo 0 > /sys/devices/platform/soc/soc:leds-gpios/leds/led_59/brightness
//LE1.0 Turn off
# echo 1 > /sys/devices/platform/soc/soc:leds-gpios/leds/led_59/brightness
//LE1.0 Turn on
# echo 0 > /sys/devices/platform/soc/soc:leds-gpios/leds/led_gen/brightness
//LE2.0 Turn off
# echo 1 > /sys/devices/platform/soc/soc:leds-gpios/leds/led_gen/brightness
//LE2.0 Turn on
# echo 255 > /sys/class/leds/red/brightness //LE1.0/LE2.0 Turn on
# echo 0 > /sys/class/leds/red/brightness //LE1.0/LE2.0 Turn off
# echo 255 > /sys/class/leds/green/brightness //LE1.0/LE2.0 Turn on
# echo 0 > /sys/class/leds/green/brightness //LE1.0/LE2.0 Turn off
# echo 255 > /sys/class/leds/blue/brightness //LE1.0/LE2.0 Turn on
# echo 0 > /sys/class/leds/blue/brightness //LE1.0/LE2.0 Turn off
```

2.7. Buttons

Refer to Figure 2-4 to locate the buttons.

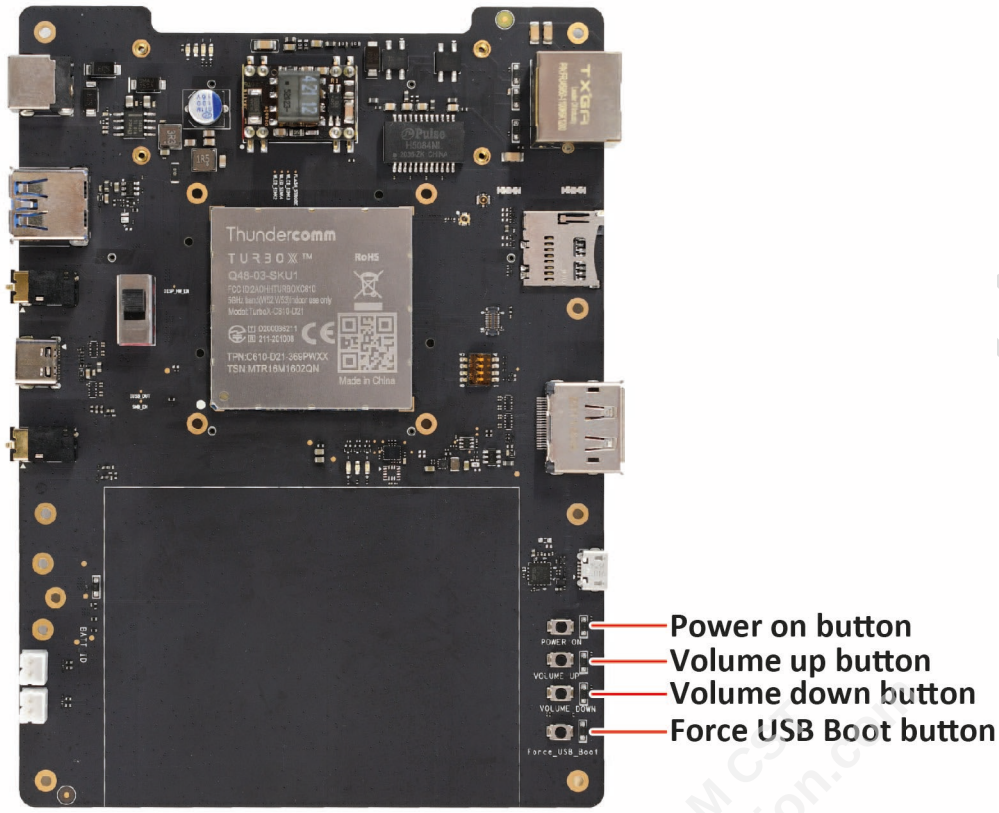


Figure 2-4.Button Layout

Table 2-1. Button description

Button	Device node
Power on	/dev/input/event0
Volume up	/dev/input/event1
Volume down	/dev/input/event0
Force USB Boot	-

➲ **NOTE**: **Force USB BOOT** is used to force the system enter emergency download mode.

To obtain the interrupt event, run the following commands:

```
$ adb root
$ adb shell
# cat dev/input/event0 | hexdump    //LE1.0/LE2.0
# cat dev/input/event1 | hexdump    //LE1.0/LE2.0
```

2.8. Wi-Fi/BT

- **Set up Wi-Fi STA mode**

1) Connect the antenna to the board via the **Wi-Fi/BT antenna connector** (No. 24 in Figure 1-1).



Figure 2-5. Wi-Fi Antenna Slot Connection

2) Run the following commands to open *mobileap_cfg.xml*.

```
$ adb root && adb shell
# vi /etc/data/mobileap_cfg.xml
```

3) Input `i` to enter edit mode and set `<WLANEnableAtBootup>` value to `1` in *mobileap_cfg.xml* to automatically enable Wi-Fi connection function at system bootup.

```
<WLANEnableAtBootup>1</WLANEnableAtBootup>
```

4) Press ESC to exit the edit mode.

5) Input `:wq!` to save and exit the modified configuration file.

6) Run the following command to open *wlan_cfg.xml*.

```
# vi /etc/data/wlan_cfg.xml
```

7) Input `i` to enter edit mode and modify `<WLANMode>` value to `STA` in *wlan_cfg.xml* to enable STA mode.

```
<WLANMode>STA</WLANMode>
```

8) Press **ESC** key to exit edit mode.

9) Input `:wq!` to save and exit the modified configuration file.

10) Execute the following command to open *wpa_supplicant.conf* and verify the connection while the device is in **Wi-Fi Station Mode**.

```
# vi /etc/misc/wifi/wpa_supplicant.conf
```

Thundercomm Confidential

11) Input `i` to enter **Edit Mode** and add information on "`ssid`" and "`psk`" of Wi-Fi as follows:

```
# Only WPA-PSK is used. Any valid cipher combination is
ctrl_interface=/var/run/wpa_supplicant
p2p_disabled=1
network={
#Open
#        ssid="example open network"
#        key_mgmt=NONE
# WPA-PSK
      ssid="zcm"                        //Input your AP's name
#     proto=WPA
#     key_mgmt=WPA-PSK
#     pairwise=TKIP CCMP
#     group=TKIP CCMP
      psk="11111111"                   //Input your AP's password
#WEP
#     ssid="example wep network"
#     key_mgmt=NONE
#     wep_key0="abcde"
#     wep_key1=0102030405
#     wep_tx_keyidx=0
}
```

12) Execute the following commands to enable **Wi-Fi Station Mode** and connect to AP network.

```
# wpa_supplicant -Dnl80211 -iwlan0 -c /etc/misc/wifi/wpa_supplicant.conf -ddddt &
# dhcpcd wlan0
```

13) Run the following command to check network connection.

```
# ifconfig wlan0
```

14) The device is successfully connected to the network with log of IP address permission displayed on your screen.

```
wlan0     Link encap:Ethernet  HWaddr 00:0A:F5:83:66:EF
          inet addr:192.168.43.92  Bcast:192.168.43.255  Mask:255.255.255.0
          inet6 addr: fe80::20a:f5ff:fe83:66ef%1736140884/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:15 errors:0 dropped:0 overruns:0 frame:0
          TX packets:18 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:3000
          RX bytes:1864 (1.8 KiB)  TX bytes:1828 (1.7 KiB)
```

15) Perform a ping test.

```
# ping www.baidu.com
PING www.baidu.com (23.59.93.51): 56 data bytes
64 bytes from 23.59.93.51: seq=0 ttl=48 time=171.722 ms
64 bytes from 23.59.93.51: seq=1 ttl=48 time=163.070 ms
64 bytes from 23.59.93.51: seq=2 ttl=48 time=244.932 ms
64 bytes from 23.59.93.51: seq=3 ttl=48 time=166.135 ms
```

➲ **NOTE:** If the device fails to perform a ping test, check the **Firewall** settings, and perform a ping test outside the **Firewall**.

- **Set up Wi-Fi AP Mode**
  - **LE1.0:**

    1) Connect the antenna to the board via the **Wi-Fi/BT antenna connector** (No. 24 in Figure 1-1).



Figure 2-6. Wi-Fi Antenna Slot Connection

    2) Run the following command to edit *mobileap_cfg.xml*.

```
$ adb shell
# vi /etc/data/mobileap_cfg.xml
  <Profile>1</Profile>
  <AutoConnect>1</AutoConnect>
  <Roaming>0</Roaming>
  ...
  <MobileAPEnableAtBootup>1</MobileAPEnableAtBootup>
  <WLANEnableAtBootup>1</WLANEnableAtBootup>
```

    ➲ **NOTES:**

    - Ensure that the `<AutoConnect>` value is `1` and `<MobileAPEnableAtBootup>` value is `1`.
    - Ensure that you set the `<WLANEnableAtBootup>` value to `1` in *mobileap_cfg.xml* to enable automatic Wi-Fi connection function at system bootup.

    3) Press **ESC** key to exit edit mode.

    4) Input `:wq!` to save and exit the modified configuration file.

    5) Run the following command to open *wlan_cfg.xml*.

```
# vi /etc/data/wlan_cfg.xml
```

    6) Input `i` to enter edit mode and modify `<WLANMode>` value to `AP` in *wlan_cfg.xml* to enable AP mode.

```
<WLANMode>AP</WLANMode>
```

    7) Press ESC to exit the edit mode.

    8) Input `:wq!` to save and exit the modified configuration file.

9) Reboot the device and execute the following commands to check *hostapd.conf*.

```
$ adb root && adb shell
# ps | grep hostapd
3383 root 0:00 /usr/sbin/hostapd -B /etc/misc/wifi/hostapd.conf -P
/var/run/data/hostapd_ssid1.pid -e /data/entropy_file
3406 root 0:00 /usr/sbin/hostapd_cli -i wlan0 -s /var/run -p /var/run/hostapd -B -a
/usr/bin/QCMAP_StaInterface
3862 root 0:00 grep hostapd
```

10) Connect your phone to Wi-Fi.

- **Default Wi-Fi Name**: AndroidAP-sdmsteppe-WPA2**.**
- **Default Password**: 123456789.

- **LE2.0:**

1) Connect the antenna to the board via the **Wi-Fi/BT antenna connector** (No. 24 in Figure 1-1).

2) Run the following commands to start AP.

```
$ adb root && adb shell
# killall wpa_supplicant && killall dnsmasq
# hostapd -ddd -B /etc/misc/wifi/hostapd.conf
# ifconfig wlan0 192.168.3.1 up
# dnsmasq -i wlan0 -1 /data/dnsmasq.leases --no-daemon --no-resolv --no-poll --
port=5353 --dhcp-range=192.168.3.100,192.168.3.200,1h &
```

3) Connect your phone to Wi-Fi.

- **Default Wi-Fi name**: AndroidAP-sdmsteppe-WPA2**.**
- **Default password:** 123456789.

- **Test the Bluetooth function**

1) Before running **btapp**, run the **btproperty** in the background (run only once at the beginning).

```
$ adb root && adb shell
# sed -i "s/BtA2dpSinkSplitEnable=false/BtA2dpSinkSplitEnable=true/g"
/etc/bluetooth/bt_app.conf
# btproperty &
# btapp
```

2) After running **btapp**, input gap_menu and press **Enter**.

```
gap_menu
***************** Menu ******************
         enable
         disable
         inquiry
         cancel_inquiry
         pair<space><bt_address>       eg. pair 00:11:22:33:44:55
         unpair<space><bt_address>     eg. unpair 00:11:22:33:44:55
         inquiry_list
         bonded_list
         get_state
         get_bt_name
         get_bt_address
         set_bt_name<space><bt name>     eg. set_bt_name MDM_Fluoride
         set_le_bt_name<space><bt name>  eg. set_le_bt_name MDM_LE_Fluoride
         main_menu
        ****************************************
```

3) Input `enable` and press **Enter** to enable Bluetooth.

```
enable
killall: wcnssfilter: no process killed
killall: btsnoop: no process killed
killall: qcbtdaemon: no process killed
/bin/sh: qcbtdaemon: not found
...
Load Audio HAL competed
BT State is ON
```

4) Input `inquiry` and press **Enter** to start inquiry.

```
inquiry
Inquiry Started
Device Found details:
Found device Addr: 28:11:a5:01:00:a2
Found device Name: LE-Bose SoundSport
Device class is: 7936
Device Found details:
Found device Addr: e4:ba:d9:10:00:c9
Found device Name: 360FLY4K_00C8
Device class is: 7936
Device Found details:
Found device Addr: 28:11:a5:24:01:05
Found device Name: LE-reserved_N
Device class is: 7936
Inquiry Stopped automatically
```

➲ **NOTE**: To cancel inquiry, enter the following command while the inquiry is in progress:

```
cancel_inquiry
*************************** Inquiry List ******************************
1 6a:67:28:eb:b8:ea
2 1f:82:a7:09:b7:dc
3 66:7f:d5:d7:32:e6
*************************** End of List ******************************
Inquiry Stopped due to user input
```

5) Input `inquiry_list` and press **Enter** to check the inquiry list.

```
inquiry_list
*************************** Inquiry List******************************
LE-Bose SoundSport                        28:11:a5:01:00:a2
360FLY4K_00C8                             e4:ba:d9:10:00:c9
LE-reserved_N                             28:11:a5:24:01:05
*************************** End of List ******************************
```

6) Run the following command to pair outgoing SSP.

```
pair <mac address> 0/1/2   //LE1.0
pair <mac address>           //LE2.0
```

To accept or reject the outgoing pairing for the following example (pair e4:ba:d9:10:00:c9), type Yes or No and press **Enter**.

```
pair e4:ba:d9:10:00:c9 0
************************************************
 BT pairing request::Device iPhone::Pairing Code:: 281155
************************************************
 ** Please enter yes / no **
yes
************************************************
 Pairing state for 360FLY4K_00C8 is BONDED
************************************************
```

7) Check the bonded list.

a) Input `bonded_list` and press **Enter** to check the bonded list:

```
bonded_list
************************* Bonded Device List ************************
360FLY4K_00C8                                        a4:f1:e8:c6:2f:b4
***************************** End of List ******************************
```

b) To disconnect bonded device, input `disable` and press **Enter**.

```
disable
killall: qcbtdaemon: no process killed
 BtHfpAgMsgHandler event = 1029
killall: wcnssfilter: no process killed
 BT State is OFF
```

c) To exit from **btapp**, navigate to the main menu and enter `exit`.

2.9. Sensor

The TurboX C410/C610 Open Kit has an onboard sensor which consists of a 3D digital accelerometer and a 3D digital gyroscope.

Step 1.Use `see_workhorse` tool to detect sensor status.

```
$ adb root && adb shell
/ # see_workhorse
   see_workhorse version 1.29
   begin usta_get_sensor_list
   diag: Unable to get remote processor info. Continuing with just the local
processor
   end   usta_get_sensor_list
   por file: /data/sensors/por_spec.txt not found.
   psalt->begin() took 176 milliseconds
   get_sensors( sensor_types):
       resampler
       remote_proc_state
       registry
       sensor_temperature
       motion_detect
       gyro
       accel
       gyro_cal
       da_test
       data_acquisition_engine
       aont
       amd
       llcm
        diag_sensor
```

Step 2.Configure the sampling rate of the sensor.

```
# ssc_drva_test -sensor=accel -duration=30 -sample_rate=100
# ssc_drva_test -sensor=gyro -duration=30 -sample_rate=100
…
PASS
```

Step 3.Read real-time sampling data of the sensor.

- Accelerometer

```
# sns_hal_batch -s 1 -sr 10 -rr 10 -d 30
HAL open
HAL module_api_version: 0x1
HAL hal_api_version    : 0x256
HAL hal_id             : sensors
HAL hal_name           : QTI Sensors HAL Module
HAL hal_author         : Qualcomm Technologies, Inc.
diag: Unable to get remote processor info. Continuing with just the local
processor
get_sensors_list took 3029867603 nanoseconds
batch success
activate success
Activated sensor [Type: 1] lsm6dso Accelerometer Wakeup (d) for 30 seconds,
sampling at 10.000000 Hz and reporting at 10.000000 Hz
[Type:  1] android.sensor.accelerometer (wakeup)
  Name:lsm6dso Accelerometer Wakeup Vendor:STMicro Version:140548 Handle:1
  maxRange: 19.613300 resolution: 0.000598 power: 0.170000 mA
  minDelay: 2403 us maxDelay: 1000000 us
  fifoReservedEventCount: 3000 fifoMaxEventCount: 10000
  requiredPermission:
Sleeping for 30 seconds before deactivating and exiting
2981937.027246, android.sensor.accelerometer/lsm6dso Accelerometer Wakeup,
1635432709459.290011, 0.414278, -1.321802, 9.861372, 0.000000, -
4636120683320697777, latency(ms): 16811314346187.288851
2981937.027246, android.sensor.accelerometer/lsm6dso Accelerometer Wakeup,
1635432709559.290011, 0.416135, -1.323119, 9.860840, 0.000000, -
4636073215342080074, latency(ms): 16811314346087.288851
2982136.904330, android.sensor.accelerometer/lsm6dso Accelerometer Wakeup,
1635432709659.290011, 0.413064, -1.324163, 9.862490, 0.000000, -
4636035595723637476, latency(ms): 16811314346187.165935
```

- Gyroscope

```
# sns_hal_batch -s 4 -sr 10 -rr 10 -d 30
   HAL open
   HAL module_api_version: 0x1
   HAL hal_api_version    : 0x256
   HAL hal_id             : sensors
   HAL hal_name           : QTI Sensors HAL Module
   HAL hal_author         : Qualcomm Technologies, Inc.
   diag: Unable to get remote processor info. Continuing with just the local
processor
   get_sensors_list took 3030233540 nanoseconds
   batch success
   activate success
   Activated sensor [Type: 4] lsm6dso Gyroscope Wakeup (d) for 30 seconds,
sampling at 10.000000 Hz and reporting at 10.000000 Hz
   [Type:  4] android.sensor.gyroscope (wakeup)
       Name:lsm6dso Gyroscope Wakeup Vendor:STMicro Version:140548 Handle:5
       maxRange: 2.181597 resolution: 0.000076 power: 0.550000 mA
       minDelay: 2403 us maxDelay: 1000000 us
       fifoReservedEventCount: 0 fifoMaxEventCount: 10000
       requiredPermission:
   Sleeping for 30 seconds before deactivating and exiting
   3089705.053247, android.sensor.gyroscope/lsm6dso Gyroscope Wakeup,
1635432817159.290428, -0.003054, 0.007285, 0.001267, 0.000000,
4318589821133531916, latency(ms): 16811314346255.314435
   3089705.053247, android.sensor.additional_info/lsm6dso Gyroscope Wakeup,
1635432817159.290428, TYPE_ADDITIONAL_INFO type:0, value: 0.000000
   3089705.053247, android.sensor.additional_info/lsm6dso Gyroscope Wakeup,
1635432817159.290429, TYPE_ADDITIONAL_INFO type:65537, value: 0.000000
   3089705.053247, android.sensor.additional_info/lsm6dso Gyroscope Wakeup,
1635432817159.290430, TYPE_ADDITIONAL_INFO type:1, value: 0.000000
```

```
   3089705.053247, android.sensor.gyroscope/lsm6dso Gyroscope Wakeup,
1635432817259.290771, -0.002961, 0.007085, 0.001374, 0.000000,
4316742723202847528, latency(ms): 16811314346155.314092
   3089804.962518, android.sensor.gyroscope/lsm6dso Gyroscope Wakeup,
1635432817359.290771, -0.003253, 0.007165, 0.001374, 0.000000,
4317487822720544476, latency(ms): 16811314346155.223363
3090004.945330, android.sensor.gyroscope/lsm6dso Gyroscope Wakeup,
1635432817459.290771, -0.003418, 0.007096, 0.001280, 0.000000,
4316843878274566109, latency(ms): 16811314346255.206175
```

2.10. Ethernet

Connect an Ethernet cable to the device via the **Ethernet connector** (No. 25 in Figure 1-1).

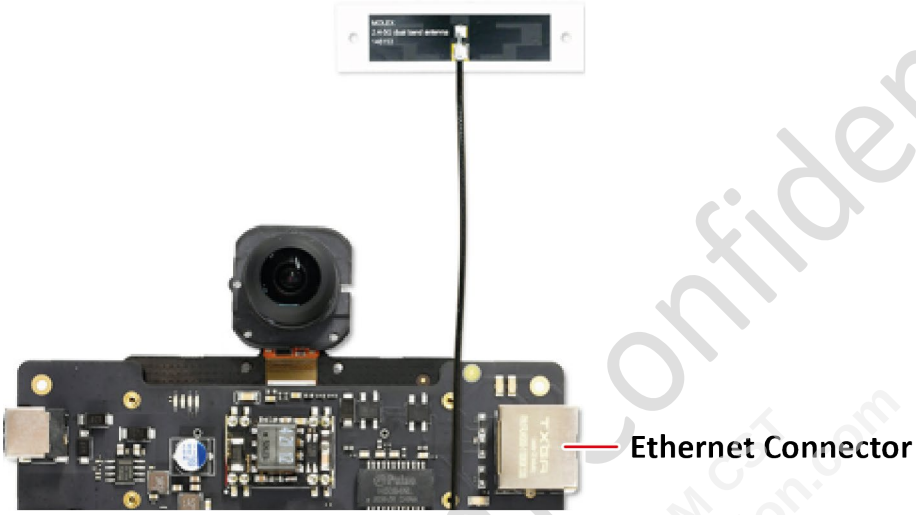


Figure 2-7. Ethernet Connection

2.10.1. WAN Mode

Step 1. `eth0` interface will be automatically created after device normal bootup.

```
$ adb root && adb shell
# ifconfig eth0 up && ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 00:55:7B:B5:7D:F7
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:121 errors:0 dropped:0 overruns:0 frame:0
          TX packets:106 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:12422 (12.1 KiB)  TX bytes:7990 (7.8 KiB)
          Interrupt:86
```

Step 2. Set up the `eth0` interface to function under **WAN Mode**.

```
# vi /etc/data/mobileap_cfg.xml
<EthBackhaul>
        <EthBackhaulMode>1</EthBackhaulMode>
</EthBackhaul>
```

Step 3. Reboot the device and the **Ethernet** will obtain the IP address.

```
$ adb root && adb shell
# ifconfig eth0
eth0      Link encap:Ethernet  Hwaddr 00:55:7B:B5:7D:F7
          inet addr:192.168.1.101  Bcast:192.168.1.255  Mask:255.255.255.0
          inet6 addr: fe80::255:7bff:feb5:7df7/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:124 errors:0 dropped:0 overruns:0 frame:0
          TX packets:124 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:13662 (13.3 KiB)  TX bytes:9924 (9.6 KiB)
          Interrupt:86
```

Step 4. Perform a ping test.

```
# ping www.baidu.com
PING www.baidu.com (220.181.38.149): 56 data bytes
64 bytes from 220.181.38.149: seq=0 ttl=48 time=8.708 ms
64 bytes from 220.181.38.149: seq=1 ttl=48 time=9.975 ms
64 bytes from 220.181.38.149: seq=2 ttl=48 time=9.404 ms
64 bytes from 220.181.38.149: seq=3 ttl=48 time=8.329 ms
^C
--- www.baidu.com ping statistics ---
4 packets transmitted, 4 packets received, 0% packet loss
round-trip min/avg/max = 8.329/9.104/9.975 ms
```

➲ **NOTE:** If the device fails to perform a ping test, check the **Firewall** settings and perform the ping test outside the **Firewall**.

2.10.2. LAN Mode

Step 1. `Eth0` interface will be automatically created after device normal bootup.

```
$ adb root && adb shell
# ifconfig eth0
eth0      Link encap:Ethernet  Hwaddr 00:55:7B:B5:7D:F7
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:121 errors:0 dropped:0 overruns:0 frame:0
          TX packets:106 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:12422 (12.1 KiB)  TX bytes:7990 (7.8 KiB)
          Interrupt:86
```

Step 2. Set up the `eth0` interface to function under LAN mode:

```
# vi /etc/data/mobileap_cfg.xml
<EthBackhaul>
        <EthBackhaulMode>0</EthBackhaulMode>
</EthBackhaul>
```

Step 3. Reboot the device and connect the device to PC via the **Ethernet connector** (No. 25 in Figure 1-1), then the device and PC will obtain an IP address automatically.

```
$ adb root && adb shell
# ifconfig eth0
eth0      Link encap:Ethernet  Hwaddr 00:55:7B:B5:7D:F7
          inet addr:169.254.4.1  Bcast:169.254.4.255  Mask:255.255.255.0
          inet6 addr: fe80::255:7bff:feb5:7df7/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:158 errors:0 dropped:0 overruns:0 frame:0
          TX packets:32 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:19196 (18.7 KiB)  TX bytes:2844 (2.7 KiB)
          Interrupt:86
```

Step 4. Perform a ping test (device to PC).

➲ **NOTE:** If the device is failed to perform a ping test, check the **Firewall** settings and perform ping test outside the **Firewall**.

2.10.3. Ethernet MAC address (for LE1.0 only)

Step 1. Create a file to store the MAC address.

```
$ adb root && adb shell
# mkdir -p /persist/factory/ethernet/
# touch /persist/factory/ethernet/emac_config.ini
```

Step 2. Add the MAC address to `emac_config.ini`.

```
11:22:33:44:55:66
```

Step 3. Verify that the MAC address has been written successfully.

```
$ adb reboot
$ adb root && adb shell
# ifconfig eth0
          Link encap:Ethernet  Hwaddr 11:22:33:44:55:66  Driver qcom-emac-dwc-eqos
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 TX bytes:0
          Interrupt:93
```

2.11. Camera



Figure 2-8. Camera Module Installation

This section introduces the **`Gstreamer`** commands for setting up multimedia functions supported on LEPDK. The **`gst-launch`** is a command for building and running a Gstreamer pipeline. The pipeline to be built and run is specified as a collection of elements and the properties are separated by an exclamation mark "!".

For more information on the **gst-launch**, refer to:
https://gstreamer.freedesktop.org/documentation/tools/gst-launch.html?gi-language=c.

➲ **NOTE**: Regarding the plug-in name "name=camsrc" of qtiqmmfsrc, some commands in the SDK FC3 version have been modified from `qmmf` to `camsrc`, which requires special attention.

2.11.1. Environment configuration

Follow the steps below to configure the environment.

➲ **NOTE**: The following steps can only be performed for one time after device flash.

Step 1. Disable */dev/dm-0 write-protected*.

```
$ adb root
$ adb disable-verity
$ adb reboot
```

Step 2. Set up the camera module.

```
$ adb root
$ adb shell mount -o remount,rw /
$ adb shell mkdir -p /etc/camera
$ adb shell touch /etc/camera/camxoverridesettings.txt
$ adb shell “echo IFEDualClockThreshold=600000000 >>
/etc/camera/camxoverridesettings.txt”
$ adb shell “echo maxHalRequests=8 >> /etc/camera/camxoverridesettings.txt”
```

Step 3. Set the display log level.

```
$ adb shell mkdir -p /data/misc/display
$ adb shell “echo 0 > /data/misc/display/gbm_dbg_cfg.txt”
$ adb shell sync
```

Step 4. Reboot the device.

```
$ adb reboot
```

Step 5. Configure the **Gstreamer** environment.

```
$ adb shell
# source /etc/gstreamer1.0/set_gst_env.sh    //Only applies to LE1.0
```

2.11.2. Push video stream to PC via adb tool

Step 1. Start to run **gst-launch-1.0** on the device.

```
$ adb root && adb remount && adb shell mount -o remount,rw /
$ adb forward tcp:8900 tcp:8900
$ adb shell
# gst-launch-1.0 -e qtiqmmfsrc name=camsrc ! video/x-
raw\(memory:GBM\),format=NV12,width=1920,height=1080,framerate=30/1 ! queue !
omxh264enc control-rate=max-bitrate target-bitrate=6000000 interval-intraframes=29
periodicity-idr=1 ! queue ! h264parse config-interval=1 ! mpegtsmux name=muxer !
queue ! tcpserversink port=8900 host=127.0.0.1
```

➲ **NOTE**: After a few seconds, the following log will display on your screen, indicating that the camera is in operation.

```
Setting pipeline to PAUSED …
Pipeline is live and does not need PREROLL …
Setting pipeline to PLAYING …
New clock:GstSystemClock
```

Step 2.Start to run **VLC-Media-Player** on a Windows/Ubuntu PC.

- **On Windows:**

  1) Open **VLC** media play and turn off **Windows Firewall**.

  2) Go to **Media** → **Open Network Stream**.

  3) Enter `tcp://127.0.0.1:8900` to get network URL.

- **On Ubuntu:**

```
$ vlc -vvv tcp://127.0.0.1:8900
```

2.11.3. Push video stream to PC via network

➲ **NOTE**: Make sure the DK and host (PC) are in the same network segment. For more information, refer to 2.10. Ethernet.

Step 1.Run an RTSP server and **gst-launch-1.0** on device.

```
$ adb root && adb remount && adb shell mount -o remount,rw /
$ adb shell
# gst-rtsp-server -a IP -p 8900 -m /live "( udpsrc name=pay0 port=8554
caps=\"application/x-rtp,media=video,clock-rate=90000,encoding-
name=H264,payload=96\" )" &
# gst-launch-1.0 -e qtiqmmfsrc name=camsrc ! video/x-
raw\(memory:GBM\),format=NV12,width=3840,height=2160,framerate=30/1 ! queue !
omxh264enc control-rate=max-bitrate target-bitrate=6000000 interval-intraframes=29
periodicity-idr=1 ! queue ! h264parse config-interval=-1 ! rtph264pay pt=96 !
udpsink host=IP port=8554
```

➲ **NOTE:** Please replace **IP** in the above commands with the IP address of the device.

Step 2.Start to run **VLC-Media-Player** on a Windows/Ubuntu PC.

- **On Windows:**

  1) Open **VLC** media play and turn off **Windows Firewall**.

  2) Go to **Media** → **Open Network Stream**.

  3) Enter *rtsp://**IP**:8900/live* to get the network URL.

- **On Ubuntu:**

```
$ vlc -vvv rtsp://IP:8900/live
```

➲ **NOTES:**

- Please replace IP in the above commands with the IP address of the device.
- VLC media player configuration: **Tools** -> **Preferences** -> **Input/Codecs** -> **Live555 stream transport** -> **RTP over RTSP(TCP)**.

Thundercomm Confidential
2025-01-31 9:16:44 AM CST
Stefan_ucom.moorvision.com

2.11.4. Live snapshot

Snapshots can be taken along with video capture.

Step 1. Run the following command to take a 4K snapshot while 1080P streaming is running.

```
# gst-pipeline-app -e qtiqmmfsrc name=camsrc ! video/x-
raw\(memory:GBM\),format=NV12,width=1920,height=1080,framerate=30/1 ! queue !
omxh264enc control-rate=max-bitrate target-bitrate=6000000 interval-intraframes=29
periodicity-idr=1 ! queue ! h264parse ! mp4mux ! queue ! filesink
location="/data/mux.mp4" camsrc.image_1 !
"image/jpeg,width=3840,height=2160,framerate=30/1" !multifilesink
location=/data/frame%d.jpg sync=true async=false
```

Step 2. After running the above command, a menu is printed out.

```
/ # gst-pipeline-app -e qtiqmmfsrc name=camsrc image_1::mode=continuous ! video/
x-raw\(memory:GBM\),format=NV12,width=640,height=480,framerate=30/1 ! queue ! om
xh264enc control-rate=max-bitrate target-bitrate=6000000 interval-intraframes=29
 periodicity-idr=1 ! queue ! h264parse ! mp4mux ! queue ! filesink location="/da
ta/mux.mp4" camsrc.image_1 ! "image/jpeg,width=3840,height=2160,framerate=30/1"
! queue ! avimux ! filesink location=/data/1080p_mjpeg.avi sync=true async=false

gbm_create_device(156): Info: backend name is: msm_drm


###################################### MENU ######################################

 ================================ Pipeline Controls ================================
   (0) NULL                      : Set the pipeline into NULL state
   (1) READY                     : Set the pipeline into READY state
   (2) PAUSED                    : Set the pipeline into PAUSED state
   (3) PLAYING                   : Set the pipeline into PLAYING state
 ================================== Other ==================================
   (p) Plugin Mode               : Choose a plugin which to control
   (q) Quit                      : Exit the application


Choose an option:
```

Figure 2-9. Menu Screen

Step 3. Follow the sub-steps below.

1) Select "**PLAYING**" option index: 3.

2) Select "**Plugin Mode**" option index: p.

3) Select "**camsrc**" option index: 11.

4) Select "**capture-image**" option index: 33.

5) Enter "**GstImageCaptureMode**" value for arg0: 1.

6) Enter "**guint**" value for arg1: 1.

Thundercomm Confidential 2025-01-31 9:16:44 AM cst Stefan_ucam.moorvision.com

2.11.5. Dual camera concurrency (1080P recording)

Use the “camera=” parameter to specify the camera to be used. In addition, dual 4K coding is not supported by hardware.

Step 1. Open a new terminal for the rear camera.

```
$ adb shell
# gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! video/x-
raw\(memory:GBM\),format=NV12,width=1920,height=1080,framerate=30/1 ! queue !
omxh264enc control-rate=max-bitrate target-bitrate=3000000 interval-intraframes=30
periodicity-idr=1 ! queue ! h264parse ! mp4mux ! queue ! filesink
location="/data/mux_1K30f_avc0.mp4"
```

Step 2. Open another terminal for the front camera.

```
$ adb shell
# gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=1 ! video/x-
raw\(memory:GBM\),format=NV12,width=1920,height=1080,framerate=30/1 ! queue !
omxh264enc control-rate=max-bitrate target-bitrate=3000000 interval-intraframes=30
periodicity-idr=1 ! queue ! h264parse ! mp4mux ! queue ! filesink
location="/data/mux_1k30f_avc1.mp4"
```

2.11.6. Digital zoom

The **QTI gst-pipeline-app** is an assistant tool that exposes the same capability as the **GStreamer's gst-launch-1.0** tool. It also provides an option for setting runtime properties of GST elements.

Refer to the Step 2 in 2.11.2. Push video stream to PC via adb tool to push streaming to the VLC of PC end via the adb tool.

Step 1. Run the following commands.

```
$ adb shell
# gst-pipeline-app -e qtiqmmfsrc name=camsrc ! video/x-
raw\(memory:GBM\),format=NV12,width=1920,height=1080,framerate=30/1 ! queue !
omxh264enc control-rate=max-bitrate target-bitrate=6000000 interval-intraframes=29
periodicity-idr=1 ! queue ! h264parse config-interval=1 ! mpegtsmux name=muxer !
queue ! tcpserversink port=8900 host=127.0.0.1
```

Step 2. Follow the sub-steps below.

1) Select “**PLAYING**” option index: 3.

2) Select “**Plugin Mode**” option index: p.

3) Select “**camsrc**” option index: 9.

4) Select “**zoom**” option index: 23.

5) Enter a value (or press **Enter** to keep the current one): <0, 0, 1920, 1080>.

Refer to the following examples of sensor mode 1920 x 1080:

- **Zoom in:**
  - For 1x zoom, the input value should be <0,0,1920,1080>
  - For 2x zoom, sensor mode width and height should be divided by 2, so the input value would be <0,0,960,540>.
- **Zoom out:**

  For 2x zoom, specify the location (x,y) as the first 2 arguments. Thus, the input value would be <480,270,960,540>. You can obtain this (x,y) value by dividing the final resolution by 2. Since this is a 2x zoom, the final resolution would be 960x540 and center of the frame would be (480,270).

2.11.7. Lens distortion correction

LDC (Lens Distortion Correction) is a process for correcting the distortion that is introduced due to fish-eye lens. It makes a straight line in a scene to be captured as a curved line due to the distortion of the lens. LDC is part of the usecase.xml, as a separate use case with output of IPE node passed to Image Warp CHI node. The Image Wrap CHI node uses the static mesh that is generated by lens calibration.

The CHI node calls the iWarp (image warping) library APIs internally.

The iWarp module is a software feature that runs on the Adreno GPU. It applies warping to the given input image based on the input static mesh. The iWarp library supports OpenGL.

Follow the steps below to complete LDC.

Step 1. Run **Weston** server at one terminal.

```
/* for LE1.0 */
$ adb shell
# . /etc/profile
root@qcs610-odk-64:/# export XDG_RUNTIME_DIR=/dev/socket/weston
root@qcs610-odk-64:/# chmod 0700 $XDG_RUNTIME_DIR
root@qcs610-odk-64:/# cd /usr/bin/
root@qcs610-odk-64:/usr/bin# ./weston --tty=1

/* for LE2.0 */
$ adb shell
# export WESTON_DISABLE_ATOMIC=1
# export XDG_RUNTIME_DIR=/run/user/root
# weston --tty=1 --idle-time=123456
```

Step 2. Run the **gstreamer** with LDC use case at another terminal.

```
/* for LE1.0 */
$ adb shell
# . /etc/profile
# chown -R qmmfsvr:qmmfsvr /dev/socket/weston

/* for LE2.0 */
# export XDG_RUNTIME_DIR=/run/user/root
# chown -R qmmfsvr:qmmfsvr /run/user/root
```

Step 3. Apply the following gst use cases and enter `ldc=TRUE` in **gstreamer**.

```
# gst-launch-1.0 -e qtiqmmfsrc name=camsrc ldc=TRUE ! video/x-
raw\(memory:GBM\),format=NV12,width=1920,height=1080,framerate=30/1 ! queue !
omxh264enc control-rate=max-bitrate target-bitrate=6000000 interval-intraframes=29
periodicity-idr=1 ! queue ! h264parse config-interval=1 ! mpegtsmux name=muxer !
queue ! tcpserversink port=8900 host=127.0.0.1
```

Step 4. Refer to the Step 2 in 2.11.2. Push video stream to PC via adb tool to play the stream via **adb**.

2.12. Video

2.12.1. Video recording

The video recorder supports single H264/H265 stream encoding with MP4 file dump.

- Record 4K video encoded by hardware H264:

```
$ adb root && adb remount && adb shell mount -o remount,rw /
$ adb shell
# gst-launch-1.0 -e qtiqmmfsrc name=camsrc ! video/x-
raw\(memory:GBM\),format=NV12,width=3840,height=2160,framerate=30/1 ! queue !
omxh264enc control-rate=max-bitrate target-bitrate=6000000 interval-intraframes=29
periodicity-idr=1 ! queue ! h264parse ! mp4mux ! queue ! filesink
location="/data/mux_4k_avc.mp4"
```

- Record 1080P video encoded by hardware H265:

```
# gst-launch-1.0 -e qtiqmmfsrc name=camsrc camera=0 ! video/x-
raw\(memory:GBM\),format=NV12,width=1920,height=1080,framerate=30/1 ! queue !
omxh265enc control-rate=max-bitrate target-bitrate=3000000 ! queue ! h265parse !
mp4mux ! queue ! filesink location="/data/FPS_30h265.mp4"
```

➲ **NOTE**: To cancel recording, press **CTRL**+**C**.

2.12.2. Multiplex decoding (for LE1.0 only)

Multi-decoder demo configures the decoder by reading the content of the configuration file, and supports RTSP URL and local file decoding.

The demo will appear on the external monitor of the DP interface.

Step 1. Configure the **Weston** environment.

1) To download *Turbox-C610-aarch64_AI_Demo_Firmware.tgz*, click here.

2) Push the *firmware package* to */data/* directory of the target device.

```
$ adb root
$ adb disable-verity && adb reboot
$ adb root && adb shell mount -o remount,rw /
$ adb push Turbox-C610-aarch64_AI_Demo_Firmware.tgz /data/
$ adb shell
# tar -zxvf /data/Turbox-C610-aarch64_AI_Demo_Firmware.tgz -C /data/
```

3) Unpack *Turbox-C610-aarch64_Weston_DP_Firmware.tgz* under the root directory to enable **weston** output to DP.

```
# tar -zxvf /data/Turbox-C610-aarch64_AI_Demo_Firmware/aarch64-weston-dp.tgz -C /
```

Step 2. Run the **multi-decoder**, and the demo video will be played on the DP screen.

```
# ./data/Turbox-C610-aarch64_AI_Demo_Firmware/weston_dp_cmd decode
```

Step 3. Exit **Weston**.

```
# killall weston
```

2.12.3. Multiplex streaming

The following gst-launch pipeline provides capture of two H264/AVC encoded video streams of 1080P resolution. One stream is stored to file while the other is streamed over the adb network (TCP).

```
$ adb root && adb remount && adb shell mount -o remount,rw /
$ adb shell
# gst-launch-1.0 -e qtiqmmfsrc name=camsrc ! video/x-
raw\(memory:GBM\),format=NV12,width=1920,height=1080,framerate=30/1 ! queue ! omxh264enc
control-rate=max-bitrate target-bitrate=6000000 interval-intraframes=29 periodicity-
idr=1 ! queue ! h264parse ! mp4mux ! queue ! filesink location="/data/mux.mp4" camsrc. !
video/x-raw\(memory:GBM\),format=NV12,width=1920,height=1080,framerate=30/1 ! queue !
omxh264enc control-rate=max-bitrate target-bitrate=6000000 interval-intraframes=29
periodicity-idr=1 ! queue ! h264parse config-interval=1 ! mpegtsmux name=muxer ! queue !
tcpserversink port=8900 host=127.0.0.1
```

➲ **NOTES**:

- The **qtiqmmfsrc** element is used to capture and encode both video streams.
- The h264parse and mp4mux elements process the buffers from the first stream and prepare them for storage.
- The **filesink** elements store the buffers from the first stream in the file.
- The h264parse and mpegtsmux elements process the buffers from the second stream and prepare them for streaming.
- The **tcpserversink** element then sends the buffers to the network. The queue element makes sure each path/track runs independently of the other. Refer to 2.11.2. Push video stream to PC via adb tool to play the stream via **adb**.

To stop the use case, press **CTRL** + **C**, pull the recorded content from device using the following **adb** pull command, and then play the content on the host PC.

```
$ adb pull /data/mux.mp4
```

2.12.4. Video file playback on LCD

Prerequisite: **Weston** must be in running status.

Step 1.Run the following command to start Weston and verify.

```
# export XDG_RUNTIME_DIR=/dev/socket/weston  //for LE1.0
# export XDG_RUNTIME_DIR=/run/user/root      // for LE2.0
# /usr/bin/weston --tty=1 --idle-time=0 &
# ps | grep weston
xxxx root      1h07 weston --idle-time=4294967
```

➲ **NOTE:** If the processing output is not displayed, refer to 2.3 DSI LCD panel.

Step 2.Run the following command to start video playback.

```
# gst-launch-1.0 filesrc location=/data/demo.mp4 ! qtdemux name=demux demux. !
queue ! h264parse ! qtivdec ! video/x-raw\(memory:GBM\),compression=ubwc !
waylandsink fullscreen=true
```

Thundercomm Confidential

➲ **NOTES**:

- The above commands use the filesrc plug-in to read the file, and are then demuxed by the qtdemux plug-in, parsed by the h264parse plug-in, and decoded by the qtivdec plug-in.
- Then, the decoded YUV buffers are rendered to LCD display by the Wayland sink plug-in, which is the client to **Weston** server.
- The /data/demo.mp4 format supports 720P, 1080P, and 4K (3840x2160) @30FPS, thus, users can record by themselves.

2.12.5. HDMI IN

Refer to Figure 2-10 to locate HDMI IN interface on the IMX415 camera module.

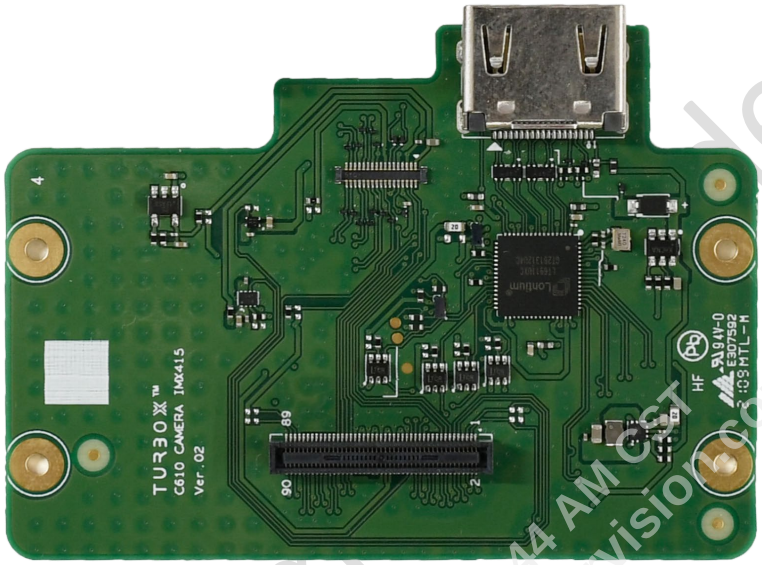


Figure 2-10. IMX415 Camera Module

Resolution supported by HDMI IN:

- 1920×1080, 60 Hz
- 3840×2160, 30 Hz

- **HDMI IN video testing method:**

  Step 1. Connect a HDMI cable to test the host PC.

  Step 2. After system bootup, check the firmware file: *lib/firmware/lt6911uxc.bin*.

```
$ adb root && adb disable-verity && adb reboot
$ adb root && adb remount && adb shell mount -o remount,rw /
$ adb shell
# mkdir -p /data/misc/display
# echo 0 >/data/misc/display/gbm_dbg_cfg.txt
# ls lib/firmware/lt6911uxc.bin
# reboot
```

Thundercomm Confidential
2025-01-31 9:16:44 AM CST
stefan_ucam.modvision.com

Step 3. Open another terminal. Test the HDMI IN audio function with the following command.

```
$ adb forward tcp:8900 tcp:8900
$ adb shell
# gst-launch-1.0 -e qtiqmmfsrc name=camsrc ! video/x-
raw\(memory:GBM\),format=NV12,width=1920,height=1080,framerate=30/1 ! queue !
omxh264enc control-rate=max-bitrate target-bitrate=6000000 interval-intraframes=29
periodicity-idr=1 ! queue ! h264parse config-interval=1 ! mpegtsmux name=muxer !
queue ! tcpserversink port=8900 host=127.0.0.1
```

Step 4. Wait for firmware upgrading which takes about 45 seconds.

Step 5. Check the log to make sure whether the upgrade is successful.

```
# adb wait-for-device logcat -b kernel | tee 1234.txt | grep -iE
"lt6911uxc|cam_qup_i2c_read"
02-07 09:50:07.437     0     0 E CAM_ERR : CAM-SENSOR: lt6911uxc_firmware_upgrade:
1256 lt6911uxc: firmware upgrade success
```

Step 6. Wait a few seconds, then open another terminal and run the following command to display the video stream.

```
# vlc -vvv tcp://127.0.0.1:8900
```

➲ **NOTE**: You can switch resolution from the source end by synchronizing the resolution parameters.

- **HDMI IN audio testing method:**

Step 1. Play music on the PC before recording. Open another terminal. Run the following commands to record the sound of HDMI IN audio.

```
$ adb root && adb disable-verity && adb reboot
$ adb root && adb remount && adb shell mount -o remount,rw /
$ adb shell
# tinymix set "MultiMedia1 Mixer TERT_MI2S_TX" "1"
# tinymix set "TERT_MI2S_TX Format" "S24_LE"
# tinymix set "TERT_MI2S_TX SampleRate" "KHZ_48"
# tinymix set "TERT_MI2S_TX Channels" "Two"
# tinycap /data/hdmiin.wav -t 60
```

Step 2. Run the command below to export *hdmiin.wav*.

```
$ adb pull /data/hdmiin.wav
```

Step 3. Play the *hdmiin.wav* file via an audio player.

Thundercomm Confidential
2025-02-31 9:18:44 AM CST
Stefan_ucammoonvision.com

2.13. Audio

2.13.1. Play back WAV

Step 1.Connect the speaker to the board via either speaker connector (No. 12 or No. 13 in Figure 1-1).

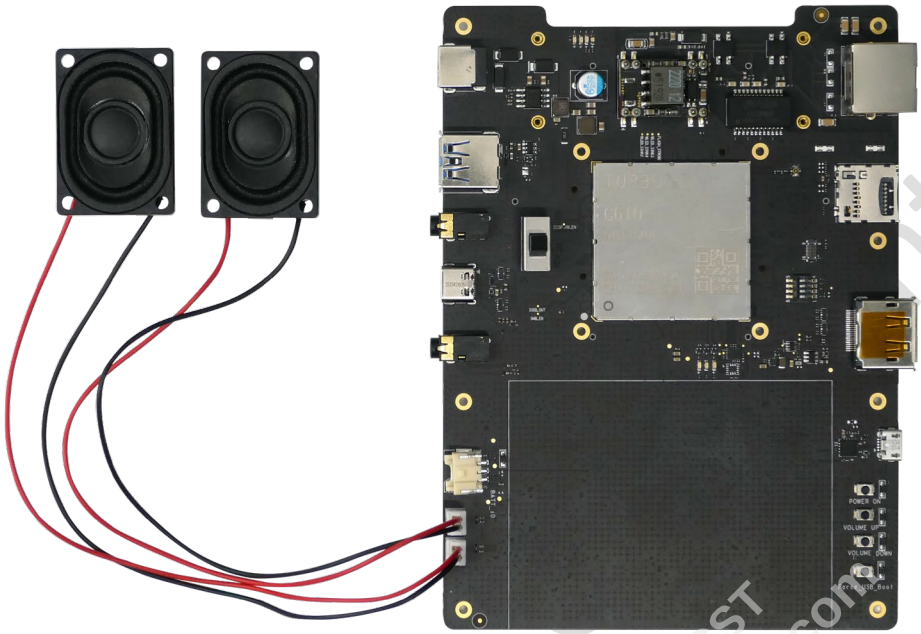


Figure 2-11. Speaker Connection

Step 2.Download the test file.

- Play back under **tinyalsa** via speaker.

  1) Push contents.

  ```
  $ adb push audiotest.wav /data/
  ```

  2) Play back with the commands below.

  ```
  $ adb shell
  # systemctl stop pulseaudio
  # tinymix set "SLIM RX0 MUX" "AIF1_PB"
  # tinymix set "CDC_IF RX0 MUX" "SLIM RX0"
  # tinymix set "SLIM RX1 MUX" "AIF1_PB"
  # tinymix set "CDC_IF RX1 MUX" "SLIM RX1"
  # tinymix set "SLIM_0_RX Channels" "Two"
  # tinymix set "RX INT7_1 MIX1 INP0" "RX0"
  # tinymix set "RX INT8_1 MIX1 INP0" "RX1"
  # tinymix set "COMP7 Switch" "1"
  # tinymix set "COMP8 Switch" "1"
  # tinymix set "SpkrLeft COMP Switch" "1"
  # tinymix set "SpkrLeft BOOST Switch" "1"
  # tinymix set "SpkrLeft VISENSE Switch" "1"
  # tinymix set "SpkrLeft SWR DAC_Port Switch" "1"
  # tinymix set "SpkrRight COMP Switch" "1"
  # tinymix set "SpkrRight BOOST Switch" "1"
  # tinymix set "SpkrRight VISENSE Switch" "1"
  # tinymix set "SpkrRight SWR DAC_Port Switch" "1"
  # tinymix set "SLIMBUS_0_RX Audio Mixer MultiMedia1" "1"
  # tinyplay /data/audiotest.wav
  ```

3) Disable the audio path.

```
$ adb shell
# tinymix set "SLIM RX0 MUX" "ZERO"
# tinymix set "CDC_IF RX0 MUX" "SLIM RX0"
# tinymix set "SLIM RX1 MUX" "ZERO"
# tinymix set "CDC_IF RX1 MUX" "SLIM RX1"
# tinymix set "SLIM_0_RX Channels" "One"
# tinymix set "RX INT7_1 MIX1 INP0" "ZERO"
# tinymix set "RX INT8_1 MIX1 INP0" "ZERO"
# tinymix set "COMP7 Switch" "0"
# tinymix set "COMP8 Switch" "0"
# tinymix set "SpkrLeft COMP Switch" "0"
# tinymix set "SpkrLeft BOOST Switch" "0"
# tinymix set "SpkrLeft VISENSE Switch" "0"
# tinymix set "SpkrLeft SWR DAC_Port Switch" "0"
# tinymix set "SpkrRight COMP Switch" "0"
# tinymix set "SpkrRight BOOST Switch" "0"
# tinymix set "SpkrRight VISENSE Switch" "0"
# tinymix set "SpkrRight SWR DAC_Port Switch" "0"
# tinymix set "SLIMBUS_0_RX Audio Mixer MultiMedia1" "0"
```

- Play back under hal_play_test_64bit via speaker

  1) Push contents.

  ```
  $ adb push audiotest.wav /data/
  ```

  2) Play back with the commands below.

  ```
  $ adb shell
  # setenforce 0
  # systemctl stop pulseaudio
  # hal_play_test_64bit -f /data/audiotest.wav -t 1 -d 2 -v 0.3 -r 48000 -c 2
  ```

- Play back under **tinyalsa** via headphones

  1) Push contents.

  ```
  $ adb push audiotest.wav /data/
  ```

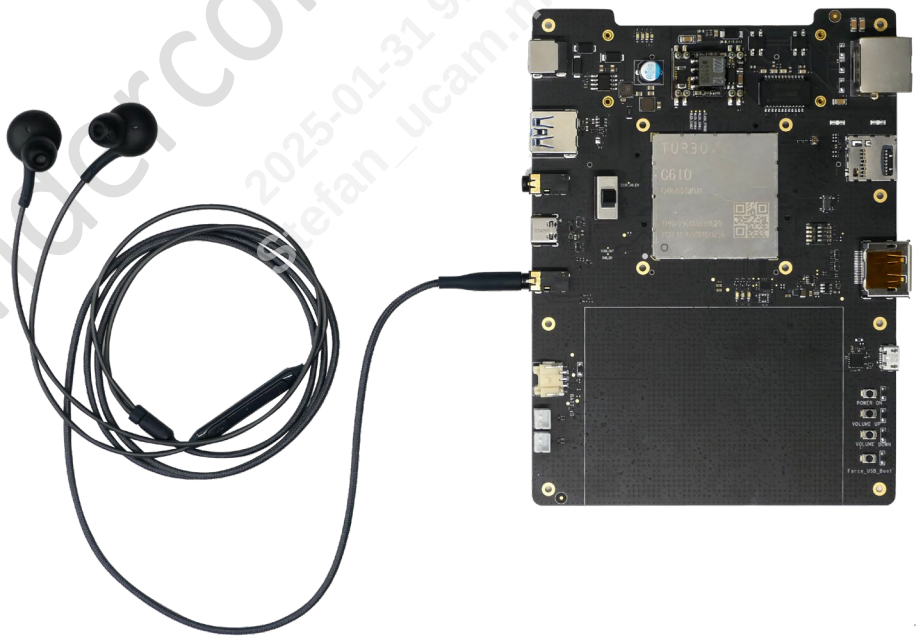


Figure 2-12. Headphone Connection

2) Play back with the commands below.

```
$ adb shell
# systemctl stop pulseaudio
# tinymix set "SLIM RX2 MUX" "AIF4_PB"
# tinymix set "SLIM RX3 MUX" "AIF4_PB"
# tinymix set "SLIM_6_RX Channels" "Two"
# tinymix set "RX INT1_2 MUX" "RX2"
# tinymix set "RX INT2_2 MUX" "RX3"
# tinymix set "SLIMBUS_6_RX Audio Mixer MultiMedia1" "1"
# tinyplay /data/audiotest.wav
```

3) Disable the audio path.

```
$ adb shell
# systemctl stop pulseaudio
# tinymix set "SLIM RX2 MUX" "ZERO"
# tinymix set "SLIM RX3 MUX" "ZERO"
# tinymix set "SLIM_6_RX Channels" "Two"
# tinymix set "RX INT1_2 MUX" "ZERO"
# tinymix set "RX INT2_2 MUX" "ZERO"
# tinymix set "SLIMBUS_6_RX Audio Mixer MultiMedia1" "0"
```

- Playback under Gstreamer via speaker

1) Push contents.

```
$ adb push audiotest.wav /data/
```

2) Play back with the commands below.

➲ **NOTE**: Make sure that you have not executed `systemctl stop pulseaudio` before the following commands; otherwise, you need to execute `systemctl start pulseaudio` or restart your device.

```
$ adb shell
# gst-launch-1.0 filesrc location=/data/audiotest.wav ! wavparse !
audioconvert ! pulsesink
```

2.13.2. Capture with DMICs

- **Capture via single DMIC**

Step 1.Capture with DMIC0.

```
$ adb shell
# systemctl stop pulseaudio
# tinymix set 'AIF1_CAP Mixer SLIM TX0' 1
# tinymix set 'CDC_IF TX0 MUX' DEC0
# tinymix set 'ADC MUX0' DMIC
# tinymix set 'DMIC MUX0' DMIC0
# tinymix set 'SLIM_0_TX Channels' One
# tinymix set 'MultiMedia1 Mixer SLIM_0_TX' 1
# tinycap /data/dmic0.wav -D 0 -d 0 -t 10 -r 48000 -c 1
```

Step 2.Capture with DMIC1.

```
$ adb shell
# systemctl stop pulseaudio
# tinymix set 'AIF1_CAP Mixer SLIM TX0' 1
# tinymix set 'CDC_IF TX0 MUX' DEC0
# tinymix set 'ADC MUX0' DMIC
# tinymix set 'DMIC MUX0' DMIC1
# tinymix set 'SLIM_0_TX Channels' One
# tinymix set 'MultiMedia1 Mixer SLIM_0_TX' 1
# tinycap /data/dmic1.wav -D 0 -d 0 -t 10 -r 48000 -c 1
```

Step 3. Disable the audio path.

```
$ adb shell
# tinymix set 'AIF1_CAP Mixer SLIM TX0' 0
# tinymix set 'CDC_IF TX0 MUX' ZERO
# tinymix set 'ADC MUX0' AMIC
# tinymix set 'DMIC MUX0' ZERO
# tinymix set 'SLIM_0_TX Channels' One
```

- **Capture via 6 DMICs**

Step 1. Capture with the commands below.

```
$ adb shell
# systemctl stop pulseaudio
# tinymix set 'AIF1_CAP Mixer SLIM TX0' 1
# tinymix set 'CDC_IF TX0 MUX' DEC0
# tinymix set 'ADC MUX0' DMIC
# tinymix set 'DMIC MUX0' DMIC0
# tinymix set 'AIF1_CAP Mixer SLIM TX1' 1
# tinymix set 'CDC_IF TX1 MUX' DEC1
# tinymix set 'ADC MUX1' DMIC
# tinymix set 'DMIC MUX1' DMIC1
# tinymix set 'AIF1_CAP Mixer SLIM TX2' 1
# tinymix set 'CDC_IF TX2 MUX' DEC2
# tinymix set 'ADC MUX2' DMIC
# tinymix set 'DMIC MUX2' DMIC2
# tinymix set 'AIF1_CAP Mixer SLIM TX6' 1
# tinymix set 'CDC_IF TX6 MUX' DEC6
# tinymix set 'ADC MUX6' DMIC
# tinymix set 'DMIC MUX6' DMIC3
# tinymix set 'AIF1_CAP Mixer SLIM TX4' 1
# tinymix set 'CDC_IF TX4 MUX' DEC4
# tinymix set 'ADC MUX4' DMIC
# tinymix set 'DMIC MUX4' DMIC4
# tinymix set 'AIF1_CAP Mixer SLIM TX5' 1
# tinymix set 'CDC_IF TX5 MUX' DEC5
# tinymix set 'ADC MUX5' DMIC
# tinymix set 'DMIC MUX5' DMIC5
# tinymix set 'MultiMedia1 Mixer SLIM_0_TX' 1
# tinymix set 'SLIM_0_TX Channels' Six
# tinycap /data/6mics.wav -D 0 -d 0 -t 10 -r 48000 -c 6
```

Step 2. Disable audio path

```
# tinymix set 'AIF1_CAP Mixer SLIM TX0' 0
# tinymix set 'CDC_IF TX0 MUX' ZERO
# tinymix set 'ADC MUX0' AMIC
# tinymix set 'DMIC MUX0' ZERO
# tinymix set 'AIF1_CAP Mixer SLIM TX1' 0
# tinymix set 'CDC_IF TX1 MUX' ZERO
# tinymix set 'ADC MUX1' AMIC
# tinymix set 'DMIC MUX1' ZERO
# tinymix set 'AIF1_CAP Mixer SLIM TX2' 0
# tinymix set 'CDC_IF TX2 MUX' ZERO
# tinymix set 'ADC MUX2' AMIC
# tinymix set 'DMIC MUX2' ZERO
# tinymix set 'AIF1_CAP Mixer SLIM TX6' 0
# tinymix set 'CDC_IF TX6 MUX' ZERO
# tinymix set 'ADC MUX6' AMIC
# tinymix set 'DMIC MUX6' ZERO
# tinymix set 'AIF1_CAP Mixer SLIM TX4' 0
# tinymix set 'CDC_IF TX4 MUX' ZERO
# tinymix set 'ADC MUX4' AMIC
# tinymix set 'DMIC MUX4' ZERO
# tinymix set 'AIF1_CAP Mixer SLIM TX5' 0
# tinymix set 'CDC_IF TX5 MUX' ZERO
# tinymix set 'ADC MUX5' AMIC
```

```
# tinymix set 'DMIC MUX5' ZERO
# tinymix set 'MultiMedia1 Mixer SLIM_0_TX' 0
# tinymix set 'SLIM_0_TX Channels' One
```

- **Play the wav file**

  Step 1. Run the commands below.

```
$ adb pull /data/dmic0.wav .
$ adb pull /data/dmic1.wav .
$ adb pull /data/6mics.wav .
```

  Step 2. Play the **.wav* file, and you can hear the recorded contents.

2.13.3. Capture with Gstreamer

➲ **NOTES**:

- Make sure that you do not modify the first channels=2;
- Make sure that you have not executed `systemctl stop pulseaudio` before the following commands; otherwise, you need to execute `systemctl start pulseaudio` or restart your device.

Step 1. The second channels=1 can be replaced by either channels=1…6 (any channel number between 1 and 6) to record single/multiple channels.

```
$ adb shell
# gst-launch-1.0 pulsesrc volume=100 ! capsfilter caps=audio/x-
raw,format=S16LE,rate=48000,channels=2 ! audioconvert ! capsfilter caps=audio/x-
raw,format=S16LE,rate=48000,channels=1 ! wavenc ! filesink location=/data/track.wav
```

Step 2. Play the wav file.

```
$ adb pull /data/track.wav .
```

Step 3. Play the **.wav* file and you can hear the recorded contents.

2.13.4. Capture with parecord

Step 1. When capturing the audio file, you can modify `--channels=1` to `--channels=1…6` (any channel between 1 and 6) to record single/multiple channels.

```
$ adb shell
# parecord --channels=1 /data/track.wav
```

Step 2. Play the wav file.

```
$ adb pull /data/track.wav .
```

Step 3. Play the **.wav*, and you can hear what you have recorded.

Thundercomm Confidential
2025-01-3 9:16:44 AM CST
stefan_ucam@noorvision.com

2.14. TensorFlow lite

Step 1.Test TensorFlow lite with label image.

```
$ adb shell mkdir -p /data/tf
```

Step 2.Obtain the model and label file.

```
$ wget
https://storage.googleapis.com/download.tensorflow.org/models/tflite/mobilenet_v1_22
4_android_quant_2017_11_08.zip
$ unzip mobilenet_v1_224_android_quant_2017_11_08.zip
$ adb push mobilenet_quant_v1_224.tflite /data/tf
$ adb push labels.txt /data/tf
```

Step 3.Obtain the sample image.

```
$ wget
https://upload.wikimedia.org/wikipedia/commons/thumb/a/ad/Commodore_Grace_M._Hopper%
2C_USN_%28covered%29.jpg/800px-Commodore_Grace_M._Hopper%2C_USN_%28covered%29.jpg
$ convert 800px-Commodore_Grace_M._Hopper\,_USN_\(covered\).jpg grace_hopper.bmp
$ adb push grace_hopper.bmp /data/tf
$ adb shell
# cd /data/tf/
# label_image -a 1 -m mobilenet_quant_v1_224.tflite -c 10
```

Step 4.The test is successful with the following information displayed on your screen.

```
Loaded model mobilenet_quant_v1_224.tflite
resolved reporter
INFO: Initialized TensorFlow Lite runtime.
Input file: ./grace_hopper.bmp
Input file size: 2400138
Input file width, height, channels: 800, 1000, 3
INFO: Created TensorFlow Lite delegate for NNAPI.

[/home/turbox/lxf/project/qcs610_2/apps_proc/build-qti-distro-fullstack-debug/tmp-
glibc/work/armv7ahf-neon-oe-linux-gnueabi/nn-framework/1.0-
r0/nn/runtime/Manager.cpp(468):(2)]DeviceManager::DeviceManager
[/local/mnt/workspace/lnxbuild/project/trees_in_use/free_tree_dir/checkout/build-
qti-distro-fullstack-perf/tmp-glibc/work/armv7ahf-neon-oe-linux-gnueabi/nnhal/1.0-
r0/nnhal/unifiedhal/driver/src/NnHalService.cpp(381):(2)]nnhal main 1.2-17
INFO: Replacing 31 node(s) with delegate (TfLiteNnapiDelegate) node, yielding 1
partitions.
Applied NNAPI delegate.invoked
average time: 12.6158 ms
0.121569: 620 lampshade
0.0745098: 495 chime
0.0745098: 410 analog clock
0.0627451: 550 envelope
0.0627451: 521 crib
```

➲ **NOTE:** For detailed information, refer to *TurboX C410 C610 Open Kit_SNPE User Manual (for LE)*.

2.15. SNPE

Step 1.Prepare before SNPE validation.

1) Download and unzip Qualcomm's SDK (1.54 or later).

2) Download and unzip snpebm.zip.

```
$ cd <SDK_project>
$ wget https://thundercomm.s3.ap-northeast-
1.amazonaws.com/uploads/web/610/snpebm.zip
$ unzip snpebm.zip
```

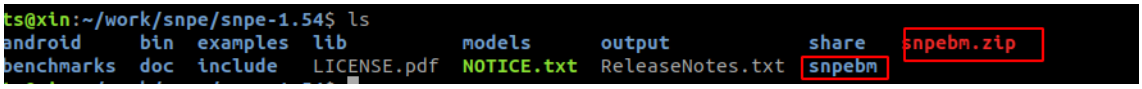


Figure 2-13. Screenshot Capture

3) Modify the following contents of *push_validate.sh* and *push_net_run.sh script*.

```
$ export SNPE_ROOT=<SDK_project>
```

➲ **NOTE:** The `<SDK_project>` is the absolute path of SNPE SDK.

Step 2. Validate SNPE runtime.

1) Run the following commands.

```
$ cd snpebm/script
$ adb root
$ ./push_validate.sh
$ adb shell
# cd /data/local/tmp/platformValidator/bin
# ./validate.sh
```

2) The following output displayed on your screen indicates that SNPE DSP runtime is ready for use.

```
PF_VALIDATOR: DEBUG: starting calculator test
PF_VALIDATOR: DEBUG: Successfully loaded DSP library –
'libcalculator_domains.so'. Setting up pointers.
PF_VALIDATOR: DEBUG: Success in executing the sum function
Runtime DSP Prerequisites: Present.
PF_VALIDATOR: DEBUG: Calling PlatformValidator->setRuntime
PF_VALIDATOR: DEBUG: CPU side validation passed.
```

Step 3. Run models on SNPE.

```
$ ./push_net_run.sh
$ adb shell
# cd /data/local/tmp/snpeexample/aarch64-ubuntu-gcc7.5/bin
# ./net_run.sh
# ./net_run.sh -r gpu
# ./net_run.sh -r dsp
```

➲ **NOTE:** If the push fails because the directory on the device does not have permissions, run the following commands:

```
$ adb root && adb disable-verity && adb reboot
$ adb root && adb remount && adb shell mount -o remount,rw /
```

Step 4. If the following information is displayed on your screen, then the test is successful.

```
--------------------------------------------------------------------------------
Model String: N/A
SNPE v1.56.2.3050
--------------------------------------------------------------------------------
Processing DNN input(s):
cropped/chairs.raw
Processing DNN input(s):
cropped/plastic_cup.raw
Processing DNN input(s):
cropped/notice_sign.raw
Processing DNN input(s):
cropped/trash_bin.raw
```

➲ **NOTE:** For detailed information, refer to *TurboX C410 C610 Open Kit_SNPE User Manual (for LE)*.

2.16. Face detection (for LE1.0 only)

The C410/C610 open kit has strong AI computing capability and can support different types of detection algorithms. The face detection algorithm sample supports the face detection function of decoded images, and draws a block diagram in the face position, showing the detection effect intuitively.

Face-detector demo configures the decoder by reading the content of the configuration file, and supports RTSP URL and local file decoding. Face-detector demo supports up to 1 channel.

To decode the local file, you can change the path to the absolute path of the file and change rtsp to 0 (example: path=/data/video.mp4, rtsp=0); to decode the RTSP stream, you can change the path to URL and change rtsp ro 1.

Refer to the following format of the configuration file:

```
[decoder_0]
cameraid=0
cameraname=camera0ne
decode=h264
enable=on
height=1080
path=rtsp://admin:1234@10.0.20.23:554
width=1920
rtsp=1
```

The demo will appear on the external monitor of the DisplayPort interface.

Step 1. Configure the **Weston** environment.

1) To Download *Turbox-C610-aarch64_AI_Demo_Firmware.tgz*, click here.

2) Push the *firmwarw package* to target device's */data/* directory.

```
$ adb root && adb shell mount -o remount,rw /
$ adb push Turbox-C610-aarch64_AI_Demo_Firmware.tgz /data/
$ adb shell
# tar -zxvf /data/Turbox-C610-aarch64_AI_Demo_Firmware.tgz -C /data/
```

3) Unpack *weston.tgz* under root directory.

```
# tar -zxvf /data/Turbox-C610-aarch64_AI_Demo_Firmware/aarch64-weston-dp.tgz -C /
```

Step 2. Run the **multi-decoder** app, and the demo video will play on the DP screen.

```
# ./data/Turbox-C610-aarch64_AI_Demo_Firmware/weston_dp_cmd face
```

Step 3. Exit **Weston**.

```
# killall weston
```

2.17. Sleep mode

Step 1.Connect the board to PC via the UART port only.

```
# echo mem > /sys/power/state
```

Step 2.Press the **Power on button** (No. 17 in Figure 1-1) to wake up board system and show kernel log with `dmesg`.

```
# dmesg
[ 1040.139909] PM: suspend entry (deep)
[ 1040.149367] PM: Syncing filesystems ... done.
[ 1040.217636] Freezing user space processes ... (elapsed 0.005 seconds) done.
[ 1040.230908] OOM killer disabled.
[ 1040.234297] Freezing remaining freezable tasks ... (elapsed 0.003 seconds) done.
[ 1040.251446] [kworker/u16:17][0x4a9de0784][02:43:49.187919] wlan: [2744:E:OSIF]
wlan_abort_scan: 1645: Failed get vdev
[ 1040.262554] [kworker/u16:17][0x4a9e148bf][02:43:49.199029] wlan: [2744:E:OSIF]
wlan_abort_scan: 1645: Failed get vdev
[ 1040.275014] [kworker/u16:17][0x4a9e4eb99][02:43:49.211440] wlan: [2744:I:HDD]
hdd_suspend_wlan: 1059: WLAN being suspended by OS
[ 1040.288109] [kworker/u16:17][0x4a9e8c5a2][02:43:49.224587] wlan: [2744:I:PMO]
pmo_core_is_wow_applicable: 323: lpass enabled, enabling wow
[ 1040.301933] [kworker/u16:1][0x4a9ecd213][02:43:49.238407] wlan: [86:I:HDD]
__wlan_hdd_bus_suspend: 789: starting bus suspend
[ 1040.313584] [kworker/u16:1][0x4a9f03c49][02:43:49.250062] wlan: [86:I:WMI]
suspend type: WOW_IFACE_PAUSE_DISABLED
[ 1040.330533] [kworker/u16:1][0x4a9f5335d][02:43:49.267010] wlan: [86:I:HDD]
__wlan_hdd_bus_suspend: 844: bus suspend succeeded
[ 1040.370598] [kworker/u16:1][0x4aa00efde][02:43:49.307071] wlan: [86:I:HDD]
__wlan_hdd_bus_suspend_noirq: 905: start bus_suspend_noirq
[ 1040.383016] [kworker/u16:1][0x4aa0493bf][02:43:49.319495] wlan: [86:I:HDD]
__wlan_hdd_bus_suspend_noirq: 946: bus_suspend_noirq done
[ 1040.396756] Disabling non-boot CPUs ...
[ 1040.428472] CPU1: shutdown
[ 1040.442634] CPU2: shutdown
[ 1040.463774] IRQ 6: no longer affine to CPU3
[ 1040.464403] CPU3: shutdown
[ 1040.484129] CPU4: shutdown
[ 1040.495777] CPU5: shutdown
[ 1040.503303] CPU6: shutdown
[ 1040.510702] CPU7: shutdown
[ 1040.517194] suspend ns:    1040517185727  suspend cycles:      20034872001
[ 1040.529404] GICv3: CPU1: found redistributor 100 region 0:0x0000000017a80000
[ 1040.529505] CPU1: Booted secondary processor [51df805e]
[ 1040.534238] CPU1 is up
[ 1040.550239] GICv3: CPU2: found redistributor 200 region 0:0x0000000017aa0000
[ 1040.550332] CPU2: Booted secondary processor [51df805e]
[ 1040.556035] CPU2 is up
[ 1040.571983] GICv3: CPU3: found redistributor 300 region 0:0x0000000017ac0000
[ 1040.572074] CPU3: Booted secondary processor [51df805e]
[ 1040.578942] CPU3 is up
[ 1040.594862] GICv3: CPU4: found redistributor 400 region 0:0x0000000017ae0000
[ 1040.594955] CPU4: Booted secondary processor [51df805e]
[ 1040.603352] CPU4 is up
[ 1040.619365] GICv3: CPU5: found redistributor 500 region 0:0x0000000017b00000
[ 1040.619457] CPU5: Booted secondary processor [51df805e]
[ 1040.629642] CPU5 is up
[ 1040.646483] GICv3: CPU6: found redistributor 600 region 0:0x0000000017b20000
[ 1040.646568] CPU6: Booted secondary processor [51df804e]
[ 1040.653245] CPU6 is up
[ 1040.669501] GICv3: CPU7: found redistributor 700 region 0:0x0000000017b40000
[ 1040.669582] CPU7: Booted secondary processor [51df804e]
[ 1040.677476] CPU7 is up

# cat /sys/power/soc_sleep/stats
aosd
```

```
        Count                    :2
        Last Entered At(sec)     :3830
        Last Exited At(sec)      :3835
        Accumulated Duration(sec):33
        Client Votes             :0x0

cxsd
        Count                    :2
        Last Entered At(sec)     :3830
        Last Exited At(sec)      :3835
        Accumulated Duration(sec):33
        Client Votes             :0x0

ddr
        Count                    :2
        Last Entered At(sec)     :3830
        Last Exited At(sec)      :3835
        Accumulated Duration(sec):33
        Client Votes             :0x0
```

2.18. OTA upgrade

This section is for LE1.0 only, please contact us to obtain OTA upgrade guide for LE2.0.

2.18.1. Full OTA upgrade

Step 1.Generate the HLOS full OTA upgrade package with the commands below:

```
# cd <path/to/workspace>
#./turbox_build.sh --zip_ota_package
```

➲ **NOTE**: The *full_update_ext4_Turbox* file in the *turbox/out* directory is the desired upgrade package.

Step 2.Generate the non-HLOS full OTA upgrade package with the commands below:

```
# cd <path/to/workspace>
#./turbox_build.sh --zip_ota_package --ota-nonhlos
```

➲ **NOTE**: The *update_ext4_ext4_Turbox* file in the *turbox/out* directory is the desired upgrade package.

Step 3.Update the full OTA image with the commands below:

```
$ adb shell "rm -fr /data/*update_*"
$ adb push full_update_ext4_Turbox-C610_xx.xx_le1.0XXX.zip /data/
$ adb shell
# env
SLOT_SUFFIX=_a
# recovery --update_package=/data/full_update_ext4_Turbox-C610_xx.xx_le1.0XXX.zip
# cat /tmp/recovery.log
…
[   18.425914] minzip: +++ processing 32768 bytes
[   18.427019] minzip: +++ processing 32768 bytes
[   18.428109] minzip: +++ processing 32768 bytes
[   18.429194] minzip: +++ processing 32768 bytes
[   18.430381] minzip: +++ processing 32768 bytes
[   18.431482] minzip: +++ processing 32768 bytes
[   18.432650] minzip: +++ processing 32768 bytes
[   18.433799] minzip: +++ processing 20480 bytes
[   18.443583] extracted boot.img to /dev/block/bootdevice/by-name/boot_b
[   18.443713] block_device_check: Checking sanity of /dev/block/bootdevice/by-
name/boot_b
[   18.443795] block_device_check: reference SHA1:
057527c5963ba5f375e17b3a7eda1621cebeaab1
[   18.509583] block_device_check: block device SHA1:
057527c5963ba5f375e17b3a7eda1621cebeaab1
[   18.509713] block_device_check: The block device SHA1 matches the reference SHA1
```

```
[   18.509993] about to run program [/usr/bin/find] with 7 args
[   18.534959] /usr/bin/find: '/WEBSERVER': Permission denied
[   19.270112] /usr/bin/find: '/system/etc/qmmf': Permission denied
[   19.555297] run_program: child exited with status 1
[   19.555370] set_inactive_slot_as_active: Setting inactive_slot(_b) as active
[  152.832753] set_inactive_slot_as_active: Set _b as active slot successfully
[  152.832940] minzip: Closing archive 0x7ff37c3c60
[  152.855207] W:Can't mount /cache/recovery/uncrypt_status
[  152.855262] I:/data/update_ext4.zip
[  152.855279] 1
[  152.855294] time_total: 152
[  152.855308] retry: 0
[  152.855323] bytes_written_system_b: 640765952
[  152.855337] bytes_stashed_system_b: 0
[  152.863434] Recovery exiting, upgrade success!
# reboot -f
$ adb shell
# env
SLOT_SUFFIX=_b
```

2.18.2. Incremental OTA upgrade

Compared to full OTA upgrade, the incremental OTA upgrade has smaller-size upgrade package, but takes longer time.

Step 1.Generate HLOS incremental OTA package.

1) Compile full image.

```
# cd <path/to/workspace>
# ./turbox_build.sh -a -l -v
```

2) Execute the following command to generate source incremental package.

```
#./turbox_build.sh --zip_ota_package
```

3) Rename the temporary incremental package *target-files-ext4-Turbox_xxxx.zip* under *<path/to/workspace>/turbox/output* to *target-files-ext4_source.zip* (that is, the current base-build on the device).

4) Make some modifications to the source code to generate differences.

5) Re-compile full image.

```
# cd <path/to/workspace>
# ./turbox_build.sh -a -l -v
```

6) Execute the following command to generate dest incremental package.

```
#./turbox_build.sh --zip_ota_package
```

7) Rename the temporary incremental package *target-files-ext4-Turbox_xxxx.zip* under *<path/to/workspace>/turbox/output* to *target-files-ext4_dest.zip.*

8) Execute the following commands to generate the OTA incremental package.

```
# cd build-qti-distro-fullstack-virtualization-debug/tmp-
glibc/deploy/images/qcs610-odk-64/ota-scripts
# export PATH=$PATH:/usr/bin:../:../../../../work/qcs610_odk_64/qti-multimedia-
image/1.0-r0/recipe-sysroot-native/usr/bin
# ./incremental_ota.sh <path/to/workspace>/turbox/output/target-files-
ext4_source.zip <path/to/workspace>/turbox/output/target-files-
ext4_dest.zip ../../../../work/qcs610_odk_64-oe-linux/qti-multimedia-image/1.0-
r0/rootfs ext4 --block --system_path /
```

The *update_incr_ext4.zip* file under *build-qti-distro-fullstack-virtualization -debug/tmp-glibc/deploy/images/qcs610-odk-64/ota-scripts* is the desired upgrade package.

Step 2.Generate non-HLOS incremental OTA package.

1) To upgrade any non-HLOS images, you can run the commands below.

```
# cd <path/to/workspace>
#./turbox_build.sh --zip_ota_package --ota-nonhlos
```

2) You will get a *target-files-ext4.zip* file containing non-HLOS images in the *<path/to/workspace>/turbox/output* directory.

3) Rename *target-files-ext4.zip* to *target-files-ext4_dest.zip*.

4) Execute Step 8 above, you will get the desired update package *update_incr_ext4.zip* that contains the modification of non-HLOS images in *build-qti-distro-fullstack-virtualization -debug/tmp-glibc/deploy/images/qcs610-odk-64/ota-scripts*.

➲ **NOTE**: The *update_incr_ext4.zip* must be based on the software version consistent with the *target_source* to guarantee successful update.

Step 3.Update the incremental OTA image.

```
$ adb shell "rm -fr /data/*update_*"
$ adb push update_incr_ext4.zip /data/
$ adb shell
# env
SLOT_SUFFIX=_a
# recovery --update_package=/data/update_incr_ext4.zip
# cat /tmp/recovery.log
…
[   38.734721] minzip: +++ processing 32768 bytes
[   38.734811] minzip: +++ processing 32768 bytes
[   38.735056] minzip: +++ processing 32768 bytes
[   38.735147] minzip: +++ processing 32768 bytes
[   38.735230] minzip: +++ processing 32768 bytes
[   38.735309] minzip: +++ processing 16348 bytes
[   38.818887] patch EMMC:/dev/block/bootdevice/by-
name/boot_a:13021184:79148812bff1d948d16c4058f455f9dc555a5a6a:13021184:d2f6f25f643d3
7a01a13150692d3c5eee76d002b: partition read matched size 13021184 sha
79148812bff1d948d16c4058f455f9dc555a5a6a
[   38.824180] 11080093696 bytes free on /cache (13021184 needed)
[   41.221534] now d2f6f25f
[   41.505685]   caches dropped
[   42.647789] verification read succeeded (attempt 1)
[   42.660285] Patching firmware images...
[   42.660353] set_inactive_slot_as_active: Setting inactive_slot(_a) as active
[   85.801824] set_inactive_slot_as_active: Set _a as active slot successfully
[   85.803057] minzip: Closing archive 0x7fd6c45880
[   85.818166] W:Can't mount /cache/recovery/uncrypt_status
[   85.818196] I:/data/update_incr_ext4.zip
[   85.818202] 1
[   85.818206] time_total: 85
[   85.818211] retry: 0
[   85.818216] bytes_written_system_a: 62308352
[   85.818220] bytes_stashed_system_a: 38420480
[   85.819471] Recovery exiting, upgrade success!
# reboot -f
$ adb shell
# env
SLOT_SUFFIX=_b
```

Appendix 1. Notices

Thundercomm may have patents or pending patent programs covering subject matter described in this document. The furnishing of this document does not give you any license to these patents. You can send license inquiries to service@thundercomm.com.

THUNDERCOMM PROVIDES THIS PUBLICATION "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE. Some jurisdictions do not allow disclaimer of express or implied warranties in certain transactions; therefore, this statement may not apply to you.

Changes are made periodically to the information herein; these changes will be incorporated in new editions of the publication. To provide better service, Thundercomm reserves the right to improve and/or modify the products and software programs described in the manuals, and the content of the manual, at any time without additional notice.

The software interface and function and hardware configuration described in the manuals included with your development board or system on module might not match exactly the actual configuration of that you have purchased. For the configuration of the product, refer to the related contract (if any) or product packing list, or consult the distributor for the product sales. Thundercomm may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

The products described in this document are not intended for use in implantation or other life support applications where malfunction may result in injury or death to persons. The information contained in this document does not affect or change Thundercomm product specifications or warranties. Nothing in this document shall operate as an express or implied license or indemnity under the intellectual property rights of Thundercomm or third parties. All information contained in this document was obtained in specific environments and is presented as an illustration. The result obtained in other operating environments may vary.

The information of this document should not be as any invitation for offer or any advice to the visitors. Please consult the professional comments from the sales consultant prior to do any actions of invest or purchase.

Thundercomm may use or distribute any of the information you supply in any way it believes appropriate without incurring any obligation to you.

Any references in this publication to non-Thundercomm Web sites are provided for convenience only and do not in any manner serve as an endorsement of those Web sites. The materials at those Web sites are not part of the materials for this Thundercomm product, and use of those Web sites is at your own risk.Thundercomm shall not be responsible for the content of the third party.

Any performance data contained herein was determined in a controlled environment. Therefore, the result obtained in other operating environments may vary significantly. Some measurements may have been made on development-level systems and there is no guarantee that these measurements will be the same on generally available systems. Furthermore, some measurements may have been estimated through extrapolation. Actual results may vary. Users of this document should verify the applicable data for their specific environment.

This document is copyrighted by Thundercomm and the property right of the date mentioned in this document, including but not limited trademarks, patents, copyrights, trade name etc. is are not covered by any open-source license. Thundercomm may update this document at any time without notice.

Anyone doesn’t have the right to amend, reprint, republication, reproduce, transmit, distribute or any other way to use this document in business or public purpose without the prior written consent by Thundercomm.

E-mail messages sent to Thundercomm via the Internet are not guaranteed to be completely secure.Thundercomm shall not be liable for any loss incurred by the surfer when transmitting any information

Thundercomm Confidential

over the Internet or for any loss incurred by Thundercomm when sending any information over the Internet at your request.

Thundercomm has all rights under other relevant exemptions provided by laws and regulations, and Thundercomm's failure to claim or delay in claiming such rights shall not be deemed to be a waiver of such rights by Thundercomm.

Thundercomm reserves the right of final interpretation of this document.

Thundercomm Confidential
2025-01-31 9:16:44 AM CST
Stefan_ucam.moorvision.com

Appendix 2. Trademarks

Thundercomm, Thundercomm Turbox, TURBOX, Thundersoft turbox are trademarks of Thundercomm Corporation or its associate companies in China and/or other countries. Intel, Intel SpeedStep, Optane, and Thunderbolt are trademarks of Intel Corporation or its subsidiaries in the U.S. and/or other countries. Microsoft, Windows, Direct3D, BitLocker, and Cortana are trademarks of the Microsoft group of companies. Mini DisplayPort (mDP), DisplayPort, and VESA are trademarks of the Video Electronics Standards Association. The terms HDMI and HDMI High-Definition Multimedia Interface are trademarks or registered trademarks of HDMI Licensing LLC in the United States and other countries. Wi-Fi, Wi-Fi Alliance, WiGig, and Miracast are registered trademarks of Wi-Fi Alliance. USB-C is a registered trademark of USB Implementers Forum. All other trademarks are the property of their respective owners.

Thundercomm Confidential
2025-01-31 9:16:44 AM CST
Stefan_ucam.moorvision.com